МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)
на тему:

«NoCode платформа конструювання вебресурсів»

Виконав студент групи ІПЗ-22-3
ЛЕУС Вадим Олександрович

Керівник роботи:
САВІЦЬКИЙ Роман Святославович

Рецензент:
ВЛАСЕНКО Олег Васильович

Житомир – 2026

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»
Зав. кафедри інженерії
програмного забезпечення
Тетяна Вакалюк
«__» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: **ЛЕУС Вадим Олександрович**
Керівник роботи: **САВІЦЬКИЙ Роман Святославович**
Тема роботи: **«NoCode платформа конструювання вебресурсів»**,
затверджена Наказом закладу вищої освіти від **«__» _____ 2026 р., №__**
Вихідні дані для роботи: **об'єкт дослідження – процес конструювання та динамічної генерації вебресурсів; предмет дослідження – методи, архітектурні підходи та програмні засоби створення NoCode платформ.**
Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

Розділ	**Консультант**	Завдання видав	Завдання прийняв
1	Савіцький Р.С.		
2	Савіцький Р.С.		
3	Савіцький Р.С.		

РЕФЕРАТ

Пояснювальна записка до випускної кваліфікаційної роботи бакалавра на тему “NoCode платформа конструювання вебресурсів” містить XX сторінок друкованого тексту. Робота ілюстрована XX рисунками та містить XX таблиць. Перелік використаних джерел налічує XX найменувань і займає XX сторінок. Пояснювальна записка також містить XX сторінок додатків. Загальний обсяг роботи становить XX сторінок.

Об’єкт дослідження – процес конструювання та динамічної генерації вебресурсів.

Предмет дослідження – методи, архітектурні підходи та програмні засоби створення NoCode платформ.

Мета роботи полягає у проєктуванні та розробці програмної системи для візуального конструювання вебсторінок, яка забезпечує автоматизацію верстки, управління контентом та розгортання вебресурсів без необхідності написання програмного коду.

У роботі використано методи порівняльного аналізу для огляду існуючих рішень, системного аналізу для формулювання вимог до системи, методи об’єктно-орієнтованого проєктування та моделювання баз даних.

Практичне значення результатів полягає у розробці готового програмного продукту. Створена NoCode платформа конструювання вебресурсів побудована за клієнт-серверною архітектурою, містить візуальний редактор. Використано гібридну модель бази даних із застосуванням формату JSON. Створена платформа надає користувачам інструментарій для створення, налаштування та публікації вебпроєктів, що знижує поріг входження у веброзробку та оптимізує часові витрати.

КЛЮЧОВІ СЛОВА: NOCODE, ВЕБРЕСУРС, ВІЗУАЛЬНИЙ РЕДАКТОР, JSON, DRAG-AND-DROP.

ЗМІСТ

РЕФЕРАТ 3
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ 5
ВСТУП 6
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ 9
1.1 Постановка задачі та огляд існуючих платформ-аналогів 9
1.2 Функціональні та нефункціональні вимоги до системи 20
1.3 Вибір і обгрунтування архітектурного рішення 25
1.4 Обгрунтування стеку технологій та апаратного забезпечення 28
Висновки до першого розділу 32
РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ 33
2.1 Моделювання варіантів використання та поведінки системи 33
2.2 Проєктування логічної структури бази даних 38
2.3 Проєктування компонентної структури та взаємодії модулів 42
2.4 Розробка основних алгоритмів роботи платформи 52
Висновки до другого розділу 58
РОЗДІЛ 3. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ 59
3.1 Програмна реалізація серверної частини та підсистеми безпеки 59
3.2 Розробка візуального редактора та клієнтського інтерфейсу 59
3.3 Тестування програмної системи 59
3.4 Керівництво користувача та розгортання платформи 59
Висновки до третього розділу 59
ВИСНОВКИ 59
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 59
ДОДАТКИ 60

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

БД – база даних

ПЗ – програмне забезпечення

СУБД – система управління базами даних

API (Application Programming Interface) – прикладний програмний інтерфейс

CMS (Content Management System) – система управління контентом

CRUD (Create, Read, Update, Delete) – базові функції управління даними (створення, читання, оновлення, видалення)

CSS (Cascading Style Sheets) – каскадні таблиці стилів

DOM (Document Object Model) – об'єктна модель документа

HTML (HyperText Markup Language) – мова гіпертекстової розмітки

HTTP/HTTPS (HyperText Transfer Protocol / Secure) – протокол передачі гіпертексту

JS (JavaScript) – високорівнева мова програмування

JSON (JavaScript Object Notation) – текстовий формат обміну даними

JWT (JSON Web Token) – відкритий стандарт для безпечної передачі інформації у вигляді JSON-об'єкта

MVC (Model-View-Controller) – архітектурний шаблон проєктування програмного забезпечення (Модель-Представлення-Контролер)

NoSQL (Not Only SQL) – підхід до побудови нереляційних баз даних

RBAC (Role-Based Access Control) – управління доступом на основі ролей

REST (Representational State Transfer) – архітектурний стиль взаємодії компонентів розподіленого застосунку в мережі

SPA (Single Page Application) – односторінковий вебзастосунок

SQL (Structured Query Language) – мова структурованих запитів до реляційних баз даних

UI (User Interface) – інтерфейс користувача

UML (Unified Modeling Language) – уніфікована мова моделювання

UX (User Experience) – досвід користувача

ВСТУП

Актуальність теми. Сучасний етап розвитку інформаційних технологій характеризується цифровізацією всіх сфер суспільного життя та бізнесу. Наявність власного онлайн-представництва є критично необхідним для будь-якої комерційної діяльності, починаючи від великих корпорацій, закінчуючи малим бізнесом та індивідуальними підприємцями. Традиційний підхід до створення вебсайтів передбачає залучення команди фахівців: дизайнерів, розробників клієнтської та серверної частин, фахівців з тестування. Цей процес вимагає написання програмного коду з нуля, що є ресурсомістким, довготривалим та високовартісним завданням.

В умовах високої конкуренції бізнесу потрібні швидкі, гнучкі та економічно ефективні рішення для запуску вебресурсів. Саме тому в останні роки спостерігається стійкий тренд на використання NoCode платформ. Концепція NoCode-розробки демократизує процес створення програмного забезпечення, дозволяючи користувачам без глибоких технічних знань конструювати повноцінні вебсторінки за допомогою візуальних інтерфейсів та технології Drag-and-Drop. Це зменшує час виходу продукту на ринок та знижує поріг входження у сферу вебконструювання.

Але розробка самих NoCode систем є складною архітектурною задачею. Перехід від жорстко заданої статичної розмітки до систем динамічної генерації вебінтерфейсів вимагає впровадження нових підходів до структурування та збереження даних. Збереження згенерованого контенту як звичайного HTML-коду призводить до втрати семантики та ускладнює подальше редагування. Тому є потреба у створенні платформ, які використовують компонентний підхід, серіалізують структуру сторінок у форматі JSON та поєднують гнучкість документо-орієнтованих моделей із надійністю реляційних баз даних. З огляду на це, розробка власної NoCode платформи конструювання вебресурсів, що базується на сучасних архітектурних патернах та оптимізованих методах збереження конфігураційних даних, є актуальним науково-практичним завданням.

Об'єкт дослідження – процес конструювання та динамічної генерації вебресурсів.

Предмет дослідження – методи, архітектурні підходи та програмні засоби створення NoCode платформ.

Мета роботи полягає у проєктуванні та розробці програмної системи для візуального конструювання вебсторінок, яка забезпечує автоматизацію верстки, управління контентом та розгортання вебресурсів без необхідності написання програмного коду.

Для досягнення поставленої мети було сформульовано та вирішено такі завдання дослідження:

- Проаналізувати предметну область, дослідити саму концепцію NoCode-розробки та провести огляд існуючих аналогів.
- Обгрунтувати вибір стеку технологій та інструментів розробки, визначити функціональні й нефункціональні вимоги до програмного продукту та апаратного забезпечення.
- Спроєктувати загальну архітектуру програмної платформи, побудувати об'єктно-орієнтовану та компонентну структуру системи.
- Спроєктувати базу даних із використанням гібридної моделі, розробити алгоритми роботи та взаємодії компонентів.
- Реалізувати NoCode платформу конструювання вебресурсів, включаючи розробку серверної частини, візуального Drag-and-Drop редактора сторінок, профілю користувача та панелі адміністратора.
- Провести тестування системи, налаштувати механізми безпеки та здійснити розгортання і демонстрацію роботи готового програмного продукту.

Методи дослідження. У роботі використано методи порівняльного та системного аналізу для огляду існуючих рішень і формування вимог до системи, методи об'єктно-орієнтованого та компонентного проєктування для розробки архітектури платформи, а також методи моделювання баз даних.

Практичне значення одержаних результатів. Створена NoCode платформа надає користувачам інтуїтивно зрозумілий інструментарій для створення, налаштування та публікації вебпроєктів, що знижує поріг входження у сферу веброзробки та оптимізує часові і фінансові витрати бізнесу. Архітектурні рішення, закладені в платформу, можуть бути масштабовані або інтегровані в інші системи.

Апробація результатів роботи. Основні теоретичні та практичні результати дослідження, що викладені у випускній кваліфікаційній роботі, доповідалися на двох науково-практичних конференціях:

Леус В. О., Савіцький Р. С. Особливості реалізації компонентного підходу при розробці систем динамічної генерації вебінтерфейсів // XIX Всеукраїнська науково-практична WEB конференція аспірантів, студентів та молодих вчених «Комп'ютерні інтелектуальні системи та мережі» (КІСМ-2026). — Кривий Ріг: Криворізький національний університет, 25-27 березня 2026 р.

Леус В. О., Савіцький Р. С. Методи структуризації та збереження даних у системах візуального конструювання вебресурсів // XI Міжнародна науково-технічна конференція «Інформаційно-комп'ютерні технології». — Житомир: Державний університет «Житомирська політехніка», 02-03 квітня 2026 р.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Постановка задачі та огляд існуючих платформ-аналогів

Аналіз предметної області та проблематики

Предметною областю даного дипломного проєкту є сфера веброзробки, а саме сегмент NoCode рішень, які надають можливість користувачам без спеціалізованої технічної підготовки створювати, розгортати повноцінні вебресурси. Розвиток цифрової економіки диктує нові правила для сучасного бізнесу: наявність представництва в глобальній мережі Інтернет стала базовою вимогою для виживання та масштабування будь-якої комерційної діяльності. Проте забезпечення цієї присутності традиційними методами стикається з низкою суттєвих перешкод.

Традиційний процес створення вебсайту є складним, багатоетапним та ресурсомістким циклом. Для реалізації навіть відносно простого корпоративного сайту чи інтернет-магазину зазвичай потрібне залучення цілої команди профільних фахівців. Спочатку UI/UX дизайнери проєктують візуальні макети та логіку взаємодії користувача з інтерфейсом. Далі Frontend-розробники перетворюють ці графічні макети у робочий код за допомогою HTML, CSS та JavaScript, забезпечуючи кросбраузерність та адаптивність під різні розміри екранів. Паралельно Backend-розробники проєктують архітектуру бази даних, пишуть серверну логіку та налаштовують серверну інфраструктуру. Завершується цей цикл етапом тестування, де виявляються та виправляються програмні дефекти.

Такий класичний підхід є дорогим і тривалим. Для малого бізнесу, приватних підприємців, фрілансерів або громадських ініціатив витрати на розробку з нуля часто є непідйомними. Крім того, будь-яка зміна на сайті, наприклад додавання нового розділу, зміна розташування інформаційного блоку, все це вимагає повторного звернення до розробників, оскільки зміна структури вимагає втручання у вихідний програмний код. Це створює залежність власника бізнесу від IT-спеціалістів та значно збільшує вартість володіння вебресурсом.

Рис. 1.1 – Порівняння традиційного циклу веброзробки та концепції NoCode

Усвідомлення цього бар'єру призвело до еволюції інструментарію веброзробки та появи концепції NoCode, тобто розробка без коду. Основною ідеєю цього підходу є абстрагування складності: рутинні технічні процеси перекладаються з людини на програмні алгоритми системи. Замість написання коду користувач взаємодіє зі зрозумілим (зазвичай графічним) інтерфейсом, візуально формуючи структуру сторінки.

Рис. 1.2 – Концептуальна схема роботи розроблюваної NoCode платформи

У рамках проєктування розроблюваної системи необхідно вирішити завдання щодо автоматизації ключових етапів створення вебресурсу. Найбільш

трудомісткою складовою є автоматизація процесу верстки інтерфейсу. У ручному режимі розробник повинен прописувати ієрархію тегів, налаштовувати CSS-класи, відступи та кольори. Система повинна автоматизувати цей процес через впровадження блокового підходу. Користувач має оперувати високорівневими абстракціями – готовими функціональними блоками (наприклад, “текстовий блок”, “картинка”, “кнопка”), а платформа у фоновому режимі автоматично трансформуватиме цей вибір у семантично правильний і валідний HTML-код.

Важливою вимогою є автоматичне забезпечення адаптивності. Ручна адаптація макета під мобільні пристрої, планшети та настільні монітори вимагає розуміння медіа-запитів та принципів гнучких сіток. Програмний комплекс повинен вирішувати цю проблему автоматично, надаючи блокам попередньо налаштовану поведінку, яка самостійно перебудовує контент залежно від роздільної здатності екрану користувача без жодного втручання з боку творця сайту.

Наступним рівнем автоматизації є управління контентом та медіаресурсами. Система повинна реалізувати принцип WYSIWYG (What You See Is What You Get – “що бачиш, те й отримаєш”). Це означає, що зміна текстового наповнення чи заміна зображень має відбуватися безпосередньо в макеті сторінки, з миттєвим відображенням результату. Додатково система має автоматизувати базову обробку медіафайлів: кадрування та оптимізацію розміру зображень для вебсередовища, виключаючи необхідність використання сторонніх графічних редакторів користувачами, які часто завантажують неоптимізовані файли надмірних розмірів.

Крім візуальної частини, система має автоматизувати структурну побудову ресурсу та роботу з динамічними даними. Створення багатосторінкових сайтів вимагає налаштування маршрутизації, генерації унікальних URL-адрес та побудови навігаційних меню. Більш того, для реалізації процесів електронної комерції платформа повинна забезпечити механізми прив'язки даних. Зміна налаштувань одного шаблонного елемента в каталозі товарів має автоматично

застосовуватися до всіх позицій, що повністю виключає необхідність ручної верстки кожної окремої картки товару.

Опираючись на проведений аналіз, головним завданням даної роботи є проєктування та розробка програмного продукту – “NoCode платформа конструювання вебресурсів”. Ця система повинна стати комплексним рішенням, де користувач без навичок програмування зможе через візуальний інтерфейс за принципом Drag-and-Drop зібрати повноцінний, адаптивний та швидкодіючий сайт. Платформа має приховати від кінцевого користувача всю технічну складність взаємодії з базами даних, серверами та кодом, надавши йому інтуїтивно зрозумілий інструментарій для реалізації його бізнес-потреб. Досягнення цієї мети вимагає глибокого дослідження архітектурних підходів до побудови візуальних редакторів, методів серіалізації DOM-дерева та організації ефективної клієнт-серверної взаємодії.

Дослідження концепції NoCode-розробки та архітектурних підходів

Стрімкий розвиток інформаційних технологій та постійне зростання вимог до швидкості розробки програмного забезпечення спричинили масове поширення концепції NoCode. У контексті веброзробки ця парадигма вимагає кардинальної зміни внутрішньої архітектури застосунків: процес написання вихідного програмного коду замінюється взаємодією користувача з графічним інтерфейсом. З інженерної точки зору це означає, що рутинні технічні процеси, такі як формування DOM-дерева, налаштування маршрутизації та транзакції до бази даних, мають бути глибоко інкапсульовані всередині ядра платформи, тоді як кінцевий користувач оперує виключно візуальними сутностями.

Ключовим механізмом взаємодії у системах візуального конструювання є технологія Drag-and-Drop. У традиційному програмуванні зміна структури документа вимагає маніпуляцій з об'єктною моделлю документа (DOM) через написання відповідних скриптів. Натомість Drag-and-Drop дозволяє користувачеві за допомогою вказівного пристрою захоплювати візуальні елементи інтерфейсу, переміщувати їх у робочій області та фіксувати у потрібному місці. З архітектурної точки зору, кожна така дія автоматично

перехоплюється системою та транслюється у математичний розрахунок нових координат або індексів масиву, що миттєво оновлює внутрішній стан системи та викликає перемальовування інтерфейсу без перезавантаження сторінки.

При проєктуванні NoCode платформ конструювання вебресурсів фундаментальним рішенням є вибір базового архітектурного підходу до формування макета сторінки. Аналіз існуючих систем візуального редагування дозволяє виділити два основні підходи: редактори вільної форми (Free-form) та сіткові або блокові редактори (Grid/Sequential).

Підхід вільної форми (Free-form), який базується на абсолютному позиціонуванні елементів, надає користувачеві максимальну свободу дій. Інтерфейс такого редактора нагадує класичні графічні редактори. Користувач може розмістити будь-який елемент у будь-якій точці екрана, задаючи його позицію за допомогою координат (X та Y) відносно вікна. Незважаючи на гнучкість, цей підхід має суттєві архітектурні недоліки. Згенерований таким чином HTML-код зазвичай є надлишковим і позбавленим семантичної структури, що негативно впливає на пошукову оптимізацію та доступність. Найбільшою проблемою вільної форми є забезпечення адаптивності. Оскільки координати елементів жорстко зафіксовані, при зміні роздільної здатності екрана елементи починають накладатися один на одного або виходити за межі видимості, що вимагає від користувача ручного переналаштування координат для кожного типу пристроїв.

Альтернативним та більш сучасним підходом є сіткова або блокова структура макета (Grid/Sequential). Ця концепція базується на природному потоці документа, що використовується у вебстандартах. Замість абсолютних координат елементи позиціонуються відносно один одного, заповнюючи доступний простір контейнерів, розташованих у вигляді послідовних горизонтальних блоків або комірок гнучкої сітки. Такий підхід ідеально підходить для розробки "NoCode платформи конструювання вебресурсів". Його головною перевагою є нативна адаптивність: контент автоматично перебудовується, стискається або переноситься на нові рядки залежно від ширини екрана клієнтського пристрою.

Крім того, блокова архітектура дозволяє генерувати чистий, семантично правильний HTML-код.

Реалізація такого підходу вимагає перегляду класичних методів веброзробки. За останні роки у сфері розробки інтерфейсів спостерігається стійкий перехід від простих статичних вебсторінок до складніших односторінкових застосунків (Single Page Applications), орієнтованих на інтерактивність та можливість гнучкого налаштування користувацького інтерфейсу. Додатковим чинником, що вплинув на актуальність розробки NoCode платформ, стала необхідність динамічного формування структури вебсторінок на основі конфігураційних даних, а не жорстко заданої розмітки.

Зазначений підхід узгоджується з концепцією побудови інтерфейсів, яка застосовується у сучасних JavaScript-бібліотеках, таких як React. Основна увага в них приділяється компонентності, повторному використанню елементів та декларативному опису UI. На практиці це дозволяє розглядати інтерфейс платформи як набір незалежних ізольованих компонентів, що можуть комбінуватися між собою залежно від вимог конкретної сторінки, яку проєктує користувач.

Разом із тим класичні підходи до верстки, за яких структура сторінки фіксується безпосередньо в HTML або JSX-коді, мають низку обмежень, які роблять їх непридатними для систем візуального конструювання. Навіть незначні зміни, наприклад перестановка блоків або коригування їхніх параметрів, за класичної розробки потребують внесення змін у програмний код розробником та повторного розгортання застосунку. У межах довготривалої підтримки проєкту це ускладнює розвиток системи та збільшує часові витрати на модифікацію інтерфейсу.

Для вирішення цієї проблеми під час розробки архітектури платформи доцільно застосувати концепцію Schema-Driven UI – інтерфейс, керований схемою. Цей архітектурний патерн передбачає повне відокремлення опису структури інтерфейсу від логіки його відображення. Замість збереження готової HTML-розмітки, конфігурація кожної сторінки сайту зберігається у вигляді

спеціалізованого формату обміну даними – JSON. Уся структура сторінки описується як ієрархічне дерево об'єктів, де кожен об'єкт містить ідентифікатор типу компонента, його унікальний ключ та набір параметрів.

Принцип роботи системи візуального конструювання, побудованої за такою концепцією, полягає у поетапній обробці конфігураційних даних. Після отримання JSON-схеми з сервера, клієнтський застосунок платформи передає її до модуля рендерингу, який рекурсивно та послідовно обробляє описані blocks. Для кожного елемента, залежно від його типу, динамічно підключається відповідний компонент системи, якому передаються параметри зі схеми. У результаті формується фінальний вигляд сторінки без необхідності жорстко задавати її структуру у вихідному коді.

Для забезпечення узгодженого зовнішнього вигляду інтерфейсу та уникнення хаосу при візуальному проєктуванні, архітектура таких систем часто спирається на принципи методології Atomic Design. Відповідно до цієї концепції, компоненти організовуються у вигляді чіткої ієрархії: від найпростіших базових елементів (таких як кнопки чи поля вводу) до складніших композицій, що формують окремі готові секції сторінки. Такий підхід дає змогу розробити для NoCode платформи обмежений, але гнучкий набір стандартизованих блоків і зберігати візуальну цілісність інтерфейсів, які створюватимуть кінцеві користувачі, гарантуючи при цьому стабільність верстки при їх взаємодії через механізми Drag-and-Drop.

Аналіз існуючих платформ-аналогів

Ринок інструментів для веброзробки є насиченим і висококонкурентним. Сегмент NoCode рішень демонструє стабільне зростання, оскільки бізнес прагне зменшити залежність від дорогих команд розробників та пришвидшити вихід цифрових продуктів на ринок. Для обгрунтування доцільності розробки власної системи та формування об'єктивних вимог до неї, було проведено аналіз існуючих рішень. Для аналізу було обрано платформи, які є лідерами ринку NoCode-розробки та використовують різні архітектурні підходи (Free-form та

Grid/Sequential). В якості базових аналогів для порівняння обрано три найбільш популярні платформи: Wix, Tilda Publishing та Webflow.

Огляд платформи Wix

Wix – це хмарна платформа для створення вебсайтів, яка історично є одним із лідерів масового ринку. Довгий час платформа базувалася виключно на підході вільної форми (Free-form), надаючи користувачам абсолютну свободу переміщення елементів.

Принцип роботи класичного редактора полягає в тому, що елементи позиціонуються за абсолютними координатами, що дозволяє розмістити їх у будь-якій точці екрана. Головною перевагою платформи є величезна бібліотека готових шаблонів, вбудований магазин додатків для розширення функціоналу та низький поріг входження для новачків.

Однак цей архітектурний підхід зумовлює суттєві недоліки. Через абсолютне позиціонування елементів генерований HTML-код є надлишковим і складним для індексації пошуковими системами. Свобода переміщення створює критичні проблеми з адаптивністю: на мобільних пристроях елементи часто накладаються один на одного, вимагаючи кропіткої ручної правки для кожного типу екранів. Крім того, платформа характеризується жорсткою прив'язкою до постачальника – користувач не має можливості експортувати чистий код сайту для розміщення на власному хостингу, залишаючись залежним від інфраструктури Wix.

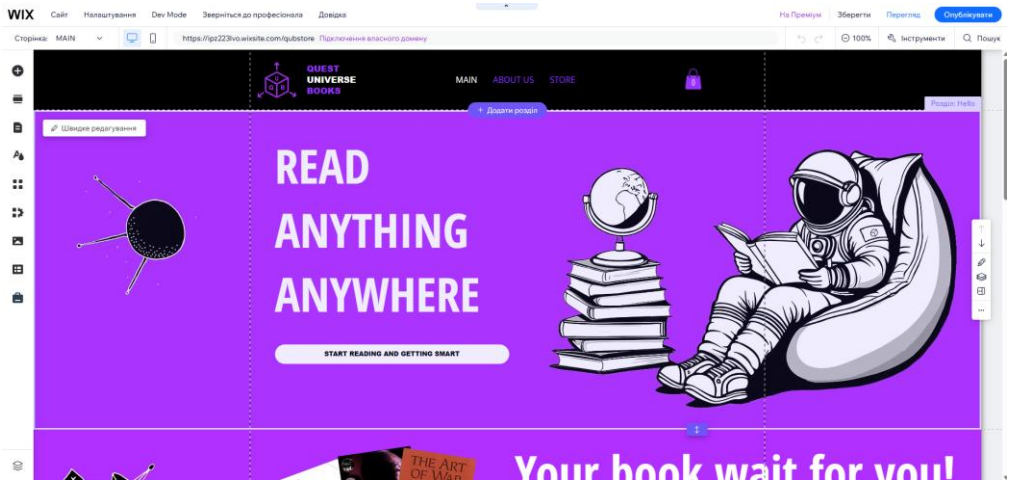

Рис. 1.3 – Інтерфейс візуального редактора платформи Wix

Огляд платформи Tilda Publishing

Tilda – це популярний конструктор сайтів, який концептуально базується на блоковому підході. Основна ідеологія платформи полягає у послідовному збиранні сторінки з готових функціональних блоків, що концептуально наближено до системи, яка розробляється у даному дипломному проєкті.

Принцип роботи полягає в тому, що кожен блок має заздалегідь визначені, фіксовані налаштування дизайну та поведінки, що гарантує гармонійний вигляд кінцевого результату. Перевагами системи є висока швидкість збірки посадкових сторінок, нативна адаптивність "з коробки" та висока естетична якість базових шаблонів.

Недоліки системи випливають з її жорсткої компонентної структури. Користувач має дуже обмежені можливості щодо кастомізації стандартних блоків. Якщо виникає потреба змінити розташування певного елемента (наприклад, перенести кнопку в іншу частину стандартного блоку), це неможливо зробити без переходу у спеціалізований "Zero Block" – складний внутрішній редактор вільної форми. Крім того, функціонал електронної комерції та можливість експорту коду доступні виключно на найдорожчих тарифних планах. Внутрішня серверна логіка магазину є базовою і не підходить для масштабування великих каталогів зі складною фільтрацією.

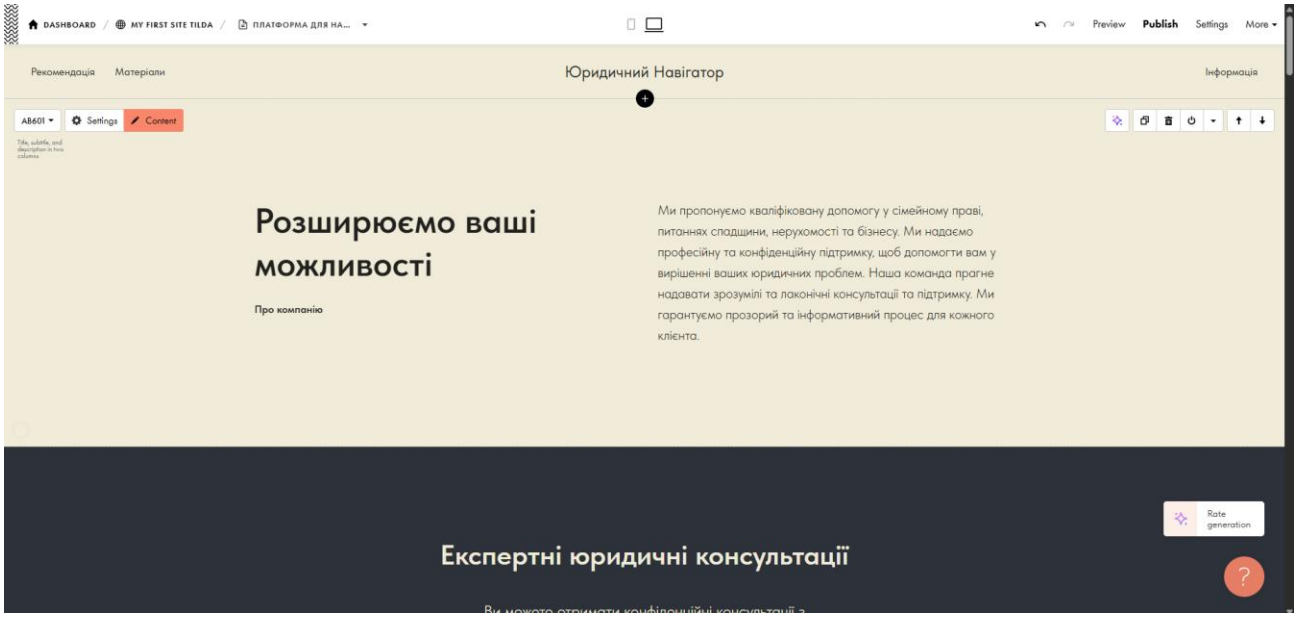

Рис. 1.4 – Інтерфейс візуального редактора платформи Tilda

Огляд платформи Webflow

Webflow – це професійний інструмент веброзробки, який є еталоном сіткового підходу та позиціонується як візуальний інтерфейс для написання коду. За рівнем складності цю платформу доцільно класифікувати як Low-Code рішення, оскільки ефективна робота в ній вимагає розуміння принципів верстки, каскадних таблиць стилів та коробкової моделі.

Інтерфейс редактора візуально нагадує інтегровані середовища розробки або складні графічні редактори. Користувач безпосередньо оперує такими поняттями, як Flexbox, Grid, Padding, Margin, Z-index. Головною перевагою Webflow є повний контроль над дизайном і складними анімаціями, генерація чистого, валідного та семантичного коду, а також наявність потужної вбудованої системи управління контентом для роботи з динамічними даними.

Проте ці переваги формують і головні недоліки системи для масового користувача. Webflow має надзвичайно високий поріг входження: людина без базових знань HTML/CSS не зможе ефективно користуватися платформою. Інтерфейс є перевантаженим для звичайного власника бізнесу або контент-менеджера. Крім того, це одне з найдорожчих рішень на ринку, що робить його недоцільним для тестування простих гіпотез або невеликих проєктів.

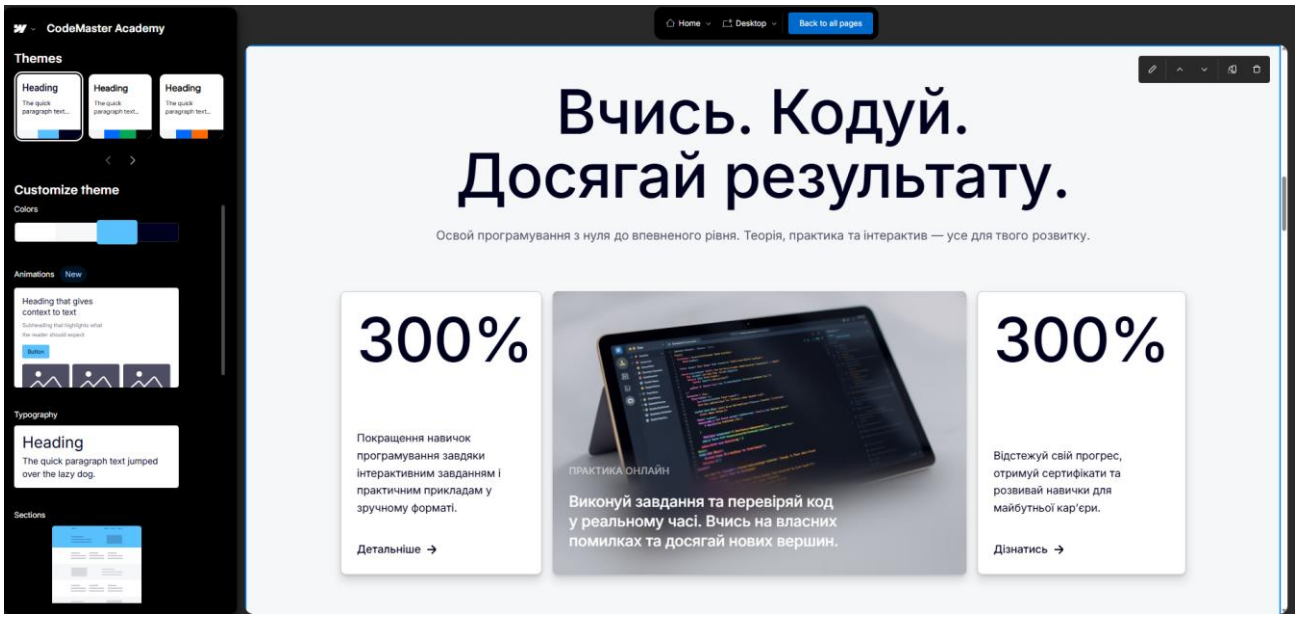

Рис. 1.5 – Інтерфейс візуального редактора платформи Webflow

Порівняльна характеристика платформ

Для наочної систематизації результатів аналізу аналогів та обгрунтування функціональних вимог до власної розробки, основні критерії порівняння зведено у таблицю (табл. 1.1).

Таблиця – 1.1 Порівняння систем конструювання вебресурсів

Критерії	**Wix**	**Tilda Publishing**	**Webflow**	**Платформа, що розробляється**
Формат збереження структури сторінок	Закритий	Закритий	Закритий	Відкритий (JSON-схема)
Наявність відкритого REST API для зовнішніх інтеграцій	Частково (Wix Velo)	Обмежено	Так	Так
Відсутність Vendor lock-in (прив'язки до вендора)	Ні	Ні	Ні	Так
Автоматичне збереження змін у фоні (Auto-save)	Так	Ні	Так	Так
Механізм Drag-and-Drop для блоків	Так	Так	Так	Так
Наскрізні глобальні компоненти (Header/Footer)	Так	Так	Так	Так
Динамічне приховування зайвих модулів (напр., вимкнення "Магазину")	Ні	Ні	Ні	Так
Наявність готових пресетів для блоків	Так	Так	Так	Так
Мультивалютність товарів та кошика	Так	Ні	Так	Так
Підтримка мультикатегорійності	Так	Так	Так	Так
Вбудований антиспам-контроль ліміту неоплачених замовлень	Ні	Ні	Ні	Так
Відстеження та управління статусами замовлень	Так	Так	Так	Так
Генерація та валідація унікальних URL-слагів для сайтів	Так	Так	Так	Так
Вбудований локальний Cookie-банер	Так	Так	Частково	Так
Наявність локальної панелі аналітики	Так	Так	Ні	Так

Висновки до огляду аналогів

Проведений аналіз продемонстрував, що існуючі комерційні рішення є суттєво поляризованими. Незважаючи на потужний функціонал існуючих рішень, усі вони мають критичні недоліки, які обмежують їх застосування для певних категорій користувачів: повністю закритий вихідний код та прив'язка до

інфраструктури постачальника, неможливість розгортання платформи на власних серверах компанії, перевантажений інтерфейс, що вимагає знань програмування (як у випадку з Webflow), або висока вартість регулярної підписки.

Тому розробка власної платформи є доцільною та науково обгрунтованою. Створювана NoCode система має зайняти проміжну нішу: вона поєднає легкість та зрозумілість компонентного підходу для новачків із генерацією чистого коду. Зберігання структури кожної сторінки у відкритому та серіалізованому JSON-форматі дозволить уникнути проблеми експорту коду, забезпечить можливість вільного експорту даних та дозволить компаніям розгортати редактор на власних серверах, зберігаючи повний контроль над своїми даними та інфраструктурою.

1.2 Функціональні та нефункціональні вимоги до системи

Аналіз цільової аудиторії та основних сценаріїв використання системи

Одним із ключових етапів проєктування програмного продукту є визначення його цільової аудиторії. Розуміння потреб кінцевих користувачів дозволяє оптимізувати користувацький інтерфейс, покращити досвід взаємодії та сфокусувати зусилля на розробці найбільш затребуваного функціоналу. Аналіз ринку NoCode рішень дозволяє виділити три основні сегменти цільової аудиторії розроблюваної платформи, кожен з яких має власні болі та специфічні вимоги до системи.

Першим сегментом є власники малого бізнесу та мікропідприємці. Це користувачі без технічної освіти, яким необхідне швидке представлення свого бізнесу в мережі Інтернет. Основна проблематика полягає в тому, що замовлення розробки сайту у вебстудії вимагає значних фінансових та часових інвестицій, а будь-яка зміна контенту у майбутньому вимагатиме залучення спеціаліста. Платформа вирішує цю проблему шляхом надання інтуїтивно зрозумілого редактора, який дозволяє зібрати сторінку з готових функціональних блоків без написання жодного рядка коду. Можливість публікації проєкту дозволяє підприємцю отримати готовий до роботи вебресурс за кілька годин.

Другий сегмент аудиторії – маркетологи та спеціалісти з реклами. Це професіонали, яким необхідно постійно тестувати нові продуктові гіпотези, створювати посадкові сторінки під різні рекламні кампанії та аналізувати трафік. Їхньою головною проблемою є залежність від IT-відділу або розробників, через що процес від створення макета до його верстки та публікації займає занадто багато часу і знижує ефективність маркетингових кампаній. Рішенням від платформи є надання можливості дублювання існуючих проєктів для проведення A/B тестування, наявність вбудованих інструментів для базової SEO-оптимізації та висока швидкість завантаження згенерованих сторінок, що позитивно впливає на конверсію.

Третій сегмент складають незалежні вебдизайнери та фрілансери, які створюють сайти на замовлення. Вони розуміються на принципах композиції та UI/UX дизайну, але не володіють глибокими знаннями у програмуванні. Проблематика полягає в тому, що більшість існуючих NoCode платформ є закритими екосистемами. Якщо клієнт хоче перенести свій сайт на власний сервер з міркувань безпеки або корпоративних політик, популярні конструктори не дозволяють цього зробити. На відміну від жорстко обмежених аналогів, розроблювана платформа вирішує це завдання шляхом підтримки функції експорту готового проєкту. Дизайнер може спроєктувати інтерфейс у візуальному редакторі, а потім вивантажити згенерований оптимізований вихідний код (статика HTML/CSS/JS) для незалежного хостингу. При цьому встановлено важливе архітектурне обмеження: система функціонує за принципом одностороннього потоку даних від редактора до коду, а функція зворотного імпорту стороннього коду в редактор свідомо не реалізовується для збереження цілісності JSON-схеми платформи.

Основні сценарії використання

На основі профілів цільової аудиторії сформовано базові сценарії взаємодії користувачів із системою, які підтверджують здатність платформи покривати різні потреби.

Сценарій швидкого запуску посадкової сторінки розпочинається з того, що користувач проходить реєстрацію та створює новий порожній проєкт. Далі, у робочому просторі візуального редактора, він послідовно додає необхідні блоки: навігаційне меню, головний екран, блок переваг, галерею та контактну форму. Після формування структури, через панель налаштувань відбувається стилізація макета шляхом зміни кольорової палітри, типографіки та завантаження власних медіафайлів. Наступним кроком користувач налаштовує SEO-параметри проєкту для коректної індексації пошуковими системами. На завершення, після візуальної перевірки, користувач натискає кнопку публікації, після чого система автоматично компілює JSON-структуру, і сайт стає доступним в мережі Інтернет.

Сценарій прототипування має інший характер взаємодії. Спочатку вебдизайнер створює макет багатосторінкового сайту-візитки для корпоративного клієнта у візуальному редакторі платформи. Після узгодження візуальної частини з клієнтом, замість стандартної публікації на інфраструктурі платформи, дизайнер ініціює процедуру експорту. Серверна частина платформи генерує архів із чистим, оптимізованим вихідним кодом проєкту, який дизайнер завантажує та самостійно розгортає на виділеному сервері клієнта.

Функціональні вимоги до системи

Функціональні вимоги описують поведінку програмної системи та визначають чіткий перелік функцій, які вона повинна виконувати для задоволення бізнес-потреб. Архітектура платформи базується на системі управління доступом на основі ролей (RBAC). У системі передбачено чотири рівні доступу: гість, авторизований користувач, модератор та адміністратор. Визначені функціональні вимоги формують чітке розмежування прав доступу і охоплюють повний життєвий цикл роботи з платформою. Детальний перелік функціональних можливостей для кожної ролі наведено у таблиці 1.2.

Таблиця 1.2 – Функціональні вимоги до системи за ролями користувачів

Роль користувача	Функціональні можливості у системі
Гість (неавторизований користувач)	Перегляд відкритого каталогу опублікованих сайтів, створених іншими користувачами платформи. Перегляд сторінок та контенту на опублікованих сайтах, взаємодія з їх публічними елементами.

Продовження таблиці 1.2

	Ознайомлення з правилами платформи. Реєстрація нового облікового запису в системі, автентифікація за допомогою електронної пошти та пароля, а також ініціація процедури безпечного відновлення доступу через підтвердження електронної пошти.
Користувач (авторизований власник)	Управління власним профілем (зміна пароля, редагування даних). Створення, редагування, копіювання та видалення власних вебсайтів. Взаємодія з візуальним редактором: конструювання структури сторінок через додавання функціональних блоків та налаштування їх компонентів (типографіки, кольорів, відступів). Збереження конфігурації у форматі JSON у базі даних. Адаптивний перегляд макетів для різних пристроїв. Завантаження та управління власними медіафайлами. Використання базового модуля електронної комерції (додавання товарів, категорій). Базове налаштування SEO. Публікація створених сайтів у відкритий доступ та збереження власних шаблонів. Створення запитів до служби підтримки та подання апеляцій щодо заблокованих ресурсів.
Модератор	Перегляд загального списку зареєстрованих користувачів, їхнього поточного статусу та переліку проєктів. Тимчасове блокування (призупинення) або розблокування доступу до сайтів порушників правил платформи, а також переведення сайтів у тіньовий бан. Винесення попереджень користувачам щодо порушення контентної політики. Видалення облікових записів порушників та пов'язаних із ними проєктів. Перегляд, взяття в обробку та надання відповідей на тікети служби підтримки. Створення глобальних шаблонів для всіх користувачів платформи та перегляд базової аналітики активності.
Адміністратор	Повний контроль над програмним комплексом, що включає всі права модератора. Доступ до повної адміністративної панелі керування платформою. Управління обліковими записами модераторів (призначення, позбавлення прав). Доступ до розширеної аналітики системи та моніторингу транзакцій.

Нефункціональні вимоги

На відміну від функціональних вимог, нефункціональні описують атрибути якості програмної системи, тобто те, як добре та швидко вона повинна виконувати свої функції. Для забезпечення високого рівня якості розробленої NoCode платформи визначено чіткі інженерні метрики.

Вимоги до продуктивності та швидкодії визначають швидкість реакції системи на дії користувача та загальну оптимізацію ресурсів. Час відгуку серверного API на стандартні запити користувачів не повинен перевищувати 200–300 мс. Оскільки візуальний редактор, побудований на базі React, є ресурсомістким клієнтським застосунком, перемальовування інтерфейсу під час активного використання технології Drag-and-Drop має відбуватися з частотою не менше 60 кадрів на секунду для забезпечення візуальної плавності. Також

згенеровані сайти користувачів повинні проходити базові метрики Google Core Web Vitals, що досягається за рахунок автоматичної оптимізації зображень через інтегрований сервіс Cloudinary та кешування статичного контенту на рівні CDN.

Вимоги до надійності та відмовостійкості спираються на обраний стек інфраструктурних рішень. Платформа повинна бути доступною цілодобово з цільовим показником експлуатаційної доступності 99,9%, що забезпечується хмарними сервісами Vercel та Render. База даних MySQL має підтримувати автоматичне резервне копіювання щонайменше один раз на добу. У разі виникнення критичної помилки на сервері, система повинна автоматично перезапускати процес за допомогою вбудованих механізмів платформи Render для мінімізації часу простою без втрати збережених даних.

Вимоги до безпеки є критично важливими через специфіку роботи з даними користувачів. Весь обмін даними між клієнтом і сервером має відбуватися виключно по захищеному протоколу HTTPS із застосуванням SSL/TLS-сертифікатів. Паролі користувачів заборонено зберігати у відкритому вигляді; вони підлягають обов'язковому хешуванню на стороні сервера. Для захисту від XSS-атак система повинна автоматично санітизувати введені користувачами дані, унеможливлюючи виконання шкідливого JavaScript-коду. Усі публічні форми також повинні бути захищені від автоматизованого перебору та спам-реєстрацій за допомогою інструментів верифікації.

Вимоги до програмного та апаратного забезпечення поділяються на дві категорії. Згенеровані сайти мають коректно та адаптивно відображатися на всіх типах сучасних пристроїв і підтримуватися актуальними версіями веббраузерів (Google Chrome 90+, Safari 14+, Firefox 88+, Microsoft Edge). Робота ж у самому візуальному редакторі через його високу складність підтримується виключно на десктопних пристроях з роздільною здатністю екрана не менше ніж 1280x720 пікселів. Апаратне забезпечення клієнта повинно мати мінімум 4 ГБ оперативної пам'яті (рекомендовано 8 ГБ), сучасний багатоядерний процесор та стабільне інтернет-з'єднання зі швидкістю не менше 5 Мбіт/с для швидкого збереження об'ємних JSON-конфігурацій.

1.3 Вибір і обгрунтування архітектурного рішення

Аналіз методів зберігання динамічних структур даних та вибір СУБД

Ефективність роботи платформи візуального конструювання критично залежить від правильно обраного підходу до зберігання даних. Архітектурний виклик полягає у необхідності одночасно забезпечувати сувору транзакційну цілісність для облікових даних користувачів та максимальну гнучкість для збереження контенту створених вебресурсів. Оскільки структура сторінки формується користувачем динамічно (невідома заздалегідь кількість блоків, їх тип та вкладеність), традиційні підходи до проєктування баз даних потребують перегляду. Під час проєктування було проаналізовано три основні патерни зберігання таких даних.

Метод 1: Патерн EAV (Entity-Attribute-Value) у реляційних базах

Історично для зберігання динамічних атрибутів у SQL-базах використовувався патерн EAV. Згідно з цим підходом, створюються три таблиці: Сутність (Блок), Атрибут (Колір, Шрифт, Відступ) та Значення. Незважаючи на гнучкість, цей підхід є антипатерном для систем рівня NoCode. Головний його недолік є катастрофічне падіння продуктивності при читанні даних. Щоб завантажити лише одну сторінку сайту, яка містить 50 блоків, кожен з яких має по 10 налаштувань стилів, базі даних доведеться виконати сотні операцій JOIN для об'єднання таблиць. Це створює надмірне навантаження на процесор сервера баз даних і робить миттєвий рендеринг сторінки неможливим.

Метод 2: Документо-орієнтовані бази даних (NoSQL)

Альтернативним рішенням є використання NoSQL баз даних, таких як MongoDB. У цій парадигмі всі дані сторінки зберігаються у вигляді єдиного BSON/JSON документа. Це ідеально підходить для дерева блоків візуального редактора, де вся конфігурація сторінки завантажується за один надшвидкий запит без жодних об'єднань. Однак використання виключно NoSQL підходу створює інші проблеми для платформи. Оскільки система передбачає модуль електронної комерції (управління товарами, замовленнями, платежами),

необхідна сувора підтримка ACID-транзакцій. Забезпечення цілісності складних зв'язків (наприклад, каскадне видалення всіх сайтів, медіафайлів та замовлень при видаленні акаунту користувача) у нереляційних базах вимагає написання складної програмної логіки на рівні бекенду, що підвищує ризик виникнення аномалій даних.

Метод 3: Гібридний підхід (MySQL з підтримкою JSON)

Для вирішення цього архітектурного компромісу у розроблюваній системі застосовано гібридний підхід. В якості основної системи управління базами даних обрано реляційну СУБД MySQL останніх версій. Вона гарантує цілісність критичних бізнес-даних (користувачі, проєкти, ролі, фінансові транзакції) через класичні таблиці зі строгими зовнішніми ключами.

Починаючи з версії 5.7, MySQL отримала нативну підтримку типу даних JSON. Це дозволяє зберігати ієрархію сторінки (дерево блоків візуального редактора) у спеціалізованій колонці block_content типу JSON всередині таблиці pages.

Переваги такого архітектурного рішення для платформи:

Швидкодія читання: Сервер отримує повну структуру сторінки для рендерингу (з усіма вкладеними блоками та їхніми налаштуваннями) єдиним простим SQL-запитом: SELECT block_content FROM pages WHERE id = ?.

Уникнення міграцій: Додавання нових функцій не вимагає зміни схеми бази даних. Схема залишається стабільною, змінюється лише наповнення JSON-документа.

Можливості пошуку: Сучасний рушій MySQL вміє індексувати віртуальні колонки, згенеровані з JSON, що дозволяє за необхідності швидко шукати специфічні дані всередині конфігурацій блоків.

Таким чином, використання реляційної моделі MySQL у комбінації з типом даних JSON надає системі гнучкість документо-орієнтованих рішень у межах надійної ACID-сумісної архітектури, що є оптимальним вибором для платформ шаблонного конструювання вебресурсів.

Інфраструктура та хмарні сервіси

Для забезпечення відмовостійкості, безпеки та безперервного розгортання платформи було обрано сучасні хмарні рішення, що дозволяють побудувати надійну архітектуру, готову до обслуговування реальних користувачів.

Як платформу для хостингу клієнтської частини (React + Vite) обрано Vercel. Цей сервіс ідеально оптимізований для сучасних фронтенд-фреймворків. Його головною перевагою є вбудована система безперервної інтеграції та розгортання – CI/CD. При надсиланні оновлень у репозиторій, Vercel автоматично збирає проєкт і оновлює вебсайт. Крім того, платформа роздає статичні файли через власну глобальну мережу доставки контенту – CDN, що забезпечує миттєве завантаження візуального редактора у користувачів з будь-якої точки світу.

Для розгортання серверної частини (Node.js) використовується хмарна платформа Render. Вона дозволяє легко розгортати вебсервіси безпосередньо з репозиторію програмного коду. Render автоматично налаштовує безпечне HTTPS-з'єднання, надає зручну панель для моніторингу логів сервера в реальному часі та автоматично перезапускає сервер у разі виникнення критичних помилок, що зводить час простою до мінімуму.

В якості хмарного провайдера для СУБД MySQL обрано платформу Aiven, яка функціонує за моделлю керованої бази даних – DBaaS (Database-as-a-Service). Такий інженерно-прагматичний підхід дозволяє уникнути витрат часу на ручне конфігурування бази даних, налаштування фаєрволів та систем резервного копіювання на Linux-сервері. Платформа Aiven бере на себе автоматичне резервне копіювання, шифрування даних у стані спокою та забезпечення високої доступності.

Для управління медіафайлами інтегровано хмарний сервіс Cloudinary. Оскільки платформа дозволяє користувачам завантажувати власні зображення для конструювання сайтів, зберігати їх локально на серверах Render або безпосередньо в базі даних є вкрай неефективним підходом. Cloudinary бере на себе автоматичну оптимізацію завантажених зображень, їх кадрування, швидку конвертацію у Web-формати та доставку через власну мережу CDN. Це критично

важливо для забезпечення швидкодії та оптимізації завантаження створених вебресурсів.

Важливою інфраструктурною вимогою для повноцінного функціонування платформи як комерційного продукту є наявність власного доменного імені. Воно необхідне не лише для ідентифікації бренду, але й для коректного налаштування політик спільного використання ресурсів між різними джерелами та роботи SSL-сертифікатів між клієнтською та серверною частинами. В якості реєстратора доменних імен обрано сервіс Namecheap через його надійність та зручні інструменти адміністрування. Визначальним фактором вибору стала безперешкодна можливість делегування управління DNS-зоною шляхом зміни NS-серверів на інфраструктуру Cloudflare.

Система мережевої безпеки та DNS-провайдер Cloudflare відіграє критичну роль у захисті застосунку. Оскільки платформа NoCode за своєю природою може бути мішенню для парсерів та зловмисних ботів, Cloudflare виступає зворотним проксі-сервером, який приховує реальні IP-адреси серверів Render та Aiven. Цей сервіс забезпечує базовий захист від DDoS-атак на мережевому (L3/L4) та прикладному (L7) рівнях, а також безкоштовно надає суворі SSL/TLS сертифікати для безпечної передачі даних.

Додатково, для забезпечення безпеки підсистеми автентифікації та захисту публічних форм платформи від автоматизованих скриптів інтегровано систему CAPTCHA Cloudflare Turnstile. Будь-який відкритий вебресурс із можливістю реєстрації піддається ризикам атак типу Brute-force (перебір паролів) та масових спам-реєстрацій. Використання цього інструменту дозволяє верифікувати легітимність запитів ще на етапі взаємодії з клієнтським застосунком. Це суттєво знижує паразитне навантаження на серверну частину та базу даних, блокуючи зловмисний трафік ще до моменту обробки бізнес-логіки.

1.4 Вибір стеку технологій та апаратного забезпечення

Вибір засобів клієнтської та серверної розробки

Вибір інструментальних засобів розробки є одним із найважливіших етапів проєктування, який безпосередньо впливає на швидкість створення програмного продукту, його надійність, продуктивність, можливості масштабування та зручність подальшої підтримки. Для реалізації проєкту було прийнято рішення використовувати стек технологій, що повністю базується на екосистемі мови JavaScript. Такий підхід дозволяє використовувати єдину мову як для клієнтської, так і для серверної частини, що суттєво зменшує час на розробку та дозволяє перевикористовувати спільний програмний код.

Вибір засобів клієнтської розробки

Клієнтська частина системи є найбільш складною складовою платформи, оскільки саме вона реалізує логіку візуального редактора, що працює безпосередньо у браузері користувача. В якості основної бібліотеки для побудови користувацького інтерфейсу було обрано React.js. Цей вибір є ідеальним для розробки NoCode-рішень з кількох вагомих причин.

По-перше, React базується на суворому компонентному підході. Це ідеально узгоджується з концепцією блокового редактора та Schema-Driven UI, які розглядались раніше. Кожен елемент майбутнього сайту інкапсулюється в окремий ізольований компонент, що значно спрощує масштабування бібліотеки блоків платформи.

По-друге, ключовою перевагою React є використання віртуальної об’єктної моделі документа – Virtual DOM. У системах візуального конструювання, де користувач постійно перетягує блоки, традиційні маніпуляції з реальним DOM є надзвичайно ресурсомісткими та викликають затримки інтерфейсу. Завдяки Virtual DOM, при переміщенні або зміні властивостей блоку, алгоритми React швидко обчислюють різницю між попереднім та новим станом, і точково перемальовують лише ті вузли на екрані, які дійсно змінилися, а не всю сторінку. Це забезпечує плавність інтерфейсу навіть при роботі зі складними та об'ємними макетами.

В якості інструменту збірки клієнтського застосунку замість традиційного Webpack було обрано Vite. Це сучасний інструмент, який розв'язує проблему

повільного запуску середовища розробки. Головним аргументом на користь Vite є миттєвий запуск локального сервера та надзвичайно швидка гаряча заміна модулів – Hot Module Replacement. Це досягається завдяки використанню нативних ES-модулів безпосередньо у браузері, що дозволяє не перезбирати весь проєкт при зміні одного файлу, а оновлювати лише змінений модуль.

Для реалізації складної механіки перетягування елементів у візуальному редакторі обрано спеціалізовану бібліотеку @dnd-kit, яка забезпечує високу продуктивність, гнучку систему детекторів колізій для точного визначення місця вставки блоку.

Вибір серверних засобів

Серверна частина відповідає за обробку API-запитів, реалізацію бізнес-логіки системи та взаємодію з базою даних. В якості програмної платформи було обрано Node.js.

Використання Node.js є виправданим для архітектури розроблюваної платформи через її асинхронну, подієво-орієнтовану модель. Оскільки NoCode платформа передбачає, що безліч користувачів можуть одночасно працювати у своїх візуальних редакторах, постійно надсилаючи на сервер масиви даних (JSON-конфігурації сайтів) для збереження, сервер повинен ефективно витримувати велику кількість одночасних підключень. Асинхронна природа Node.js дозволяє серверу не блокувати потік виконання під час очікування запису в базу даних, що робить його ідеальним вибором для систем з високим навантаженням на введення-виведення.

В якості вебфреймворку для Node.js використовується Express.js. Це мінімалістичний та гнучкий інструмент, який забезпечує швидку та зручну маршрутизацію запитів, побудову архітектури REST API та легку інтеграцію проміжного програмного забезпечення для автентифікації користувачів, логування подій та обробки помилок.

Інструментальні засоби розробки

Для забезпечення високої продуктивності процесу написання коду та проєктування архітектури системи було підібрано відповідний інструментарій розробника.

Основним інтегрованим середовищем розробки виступає **Visual Studio Code**. Цей вибір зумовлений легковажністю, високою швидкістю роботи та екосистемою розширень. Інструмент надає можливості для автодоповнення коду, інтеграцію з лінтерами для автоматичної перевірки синтаксису, а також має вбудовану підтримку роботи з терміналом та системою контролю версій Git, що оптимізує рутинні процеси.

Для проєктування, розробки та адміністрування бази даних використовується **MySQL Workbench**. Цей інструмент надає засоби для створення структур баз даних. Його використання дозволяє швидко генерувати SQL-скрипти, проводити пряме та зворотне проєктування, що є критично важливим інструментом для побудови ER-діаграм, необхідних для документації зв'язків між сутностями системи.

Для моделювання архітектури програмного забезпечення на етапі логічного проєктування обрано інструмент **PlantUML**. Діаграми не малюються графічно, а описуються за допомогою спеціалізованої текстової мови. Це дозволяє зберігати архітектурні моделі у репозиторії Git разом із вихідним кодом проєкту, відстежувати історію змін та легко коригувати діаграми без необхідності використання важких графічних пакетів.

Вимоги до апаратного забезпечення клієнта

Оскільки клієнтський застосунок, створений на базі React та Vite, переносить значну частину обчислювального навантаження (рендеринг структури, управління станом) на браузер, комп'ютер користувача має відповідати таким характеристикам:

Мінімальний обсяг оперативної пам'яті пристрою клієнта для комфортної роботи у візуальному редакторі становить 4 ГБ (рекомендований обсяг 8 ГБ).

Наявність сучасного багатоядерного процесора.

Стабільне інтернет-з'єднання зі швидкістю не менше 5 Мбіт/с для забезпечення збереження об'ємних JSON-конфігурацій та швидкого завантаження медіафайлів.

Висновки до першого розділу

У першому розділі було проведено детальний аналіз предметної області та обгрунтовано актуальність створення NoCode платформи для конструювання вебресурсів. Дослідження існуючих на ринку аналогів дозволило виявити їхні сильні та слабкі сторони, а також сформувати унікальну нішу для розроблюваного продукту. Окрім цього, було визначено ключові сегменти цільової аудиторії (мікропідприємці, маркетологи, незалежні вебдизайнери) та формалізовано базові сценарії використання системи: від швидкого запуску посадкових сторінок на піддомені платформи до прототипування з подальшим експортом чистого коду.

На основі проведеного аналізу було сформульовано головне завдання дослідження та обгрунтовано вибір технологічного стеку і методів зберігання даних. Доведено, що для забезпечення балансу між транзакційною цілісністю та гнучкістю контенту оптимальним є гібридний архітектурний підхід: використання реляційної СУБД MySQL із нативною підтримкою формату JSON замість неефективних EAV-моделей чи чистих NoSQL-рішень. Визначено єдину мову розробки JavaScript (React.js для клієнтського інтерфейсу, Node.js для серверної бізнес-логіки), а також надійну хмарну інфраструктуру (Vercel, Render, Aiven, Cloudflare).

Також у розділі було чітко визначено функціональні вимоги на основі моделі управління доступом, які розмежовують можливості гостей, користувачів, модераторів та адміністраторів. Сформульовано конкретні нефункціональні інженерні метрики, що стосуються швидкодії, безпеки, відмовостійкості та апаратного забезпечення. Зібрані вимоги, проаналізовані сценарії та обрані технології створюють міцну концептуальну основу для наступного етапу роботи – логічного та фізичного проєктування архітектури програмної системи.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

2.1 Моделювання варіантів використання та поведінки системи

Моделювання варіантів використання є одним із ключових етапів об'єктно-орієнтованого проєктування. Воно дозволяє визначити межі програмного комплексу, ідентифікувати основних дійових осіб та формалізувати сценарії їхньої взаємодії з системою. Цей етап перетворює текстові функціональні вимоги, визначені у першому розділі, у візуальну архітектуру поведінки.

На основі розробленої моделі управління доступом на основі ролей, в архітектурі платформи було виділено 4 незалежних акторів. Детальний опис кожної ролі наведено у таблиці 2.1.

Таблиця – 2.1 Ідентифікація акторів системи

Актор	Опис ролі та зона відповідальності
Гість	Неавторизований відвідувач. Має базовий доступ до публічної частини платформи, може переглядати створені сайти та ініціювати реєстрацію.
Користувач	Авторизований користувач. Має доступ до робочого простору, конструктора сайтів, управління медіафайлами та модулем електронної комерції.
Модератор	Співробітник служби підтримки. Працює в адміністративному просторі, обробляє запити користувачів, слідкує за дотриманням правил платформи та модерює контент.
Адміністратор	Технічний керівник платформи. Має найвищий рівень доступу до всіх підсистем, включаючи панель керування платформою, логування дій та фінансові транзакції.

Зважаючи на масштаб платформи, усі варіанти використання системи логічно розділені на три функціональні зони: публічний простір, спільний робочий простір та простір адміністрування.

Ключові варіанти використання у публічному та особистому просторі:

- **Реєстрація та Авторизація:** створення нового облікового запису або вхід до існуючого (Гість).
- **Перегляд правил платформи:** ознайомлення з політикою використання системи (Гість, Користувач).
- **Перегляд каталогу та пошук сайтів:** взаємодія з відкритим реєстром опублікованих ресурсів (Гість, Користувач).
- **Подача скарг на сайти:** можливість повідомити про порушення на опублікованому ресурсі (Гість, Користувач).

- **Взаємодія з E-commerce:** використання функціоналу кошика та оформлення замовлень на сайтах-магазинах (Користувач).
- **Звернення до підтримки:** створення тікетів та подача апеляцій на блокування (Користувач).

Ключові варіанти використання у робочому просторі (Спільному): Ці прецеденти доступні Користувачам для їхніх проєктів, а також Модераторам і Адміністраторам для створення сайтів чи службових шаблонів.

- **Створення та перегляд сайтів:** ініціалізація нових проєктів та зміна їхнього статусу (приватний, опубліковано).
- **Використання редактору сайтів:** взаємодія з Drag-and-Drop конструктором.
- **Завантаження медіафайлів:** управління персональною галереєю зображень.
- **Створення особистих шаблонів:** збереження напрацювань для подальшого перевикористання.
- **Налаштування профілю:** зміна особистих даних, пароля та аватара.
- **Вихід з акаунта:** безпечне завершення сесії.

Ключові варіанти використання у просторі адміністрування:

- **Модерація таблиць:** перегляд і управління статусами користувачів та їхніх сайтів (Модератор, Адміністратор).
- **Створення публічних шаблонів:** розробка базових макетів, які будуть доступні всім користувачам платформи (Модератор, Адміністратор).
- **Обробка тікетів та скарг:** комунікація з клієнтами та вирішення конфліктних ситуацій (Модератор, Адміністратор).
- **Перегляд Дашборду платформи:** аналітика загальної активності в системі (Модератор, Адміністратор).
- **Панель керування платформою:** доступ до системних логів, управління транзакціями та призначення/призупинення модераторів (тільки Адміністратор).

На основі виявлених акторів та формалізованих прецедентів побудовано розширену діаграму варіантів використання системи (рис. 2.2).

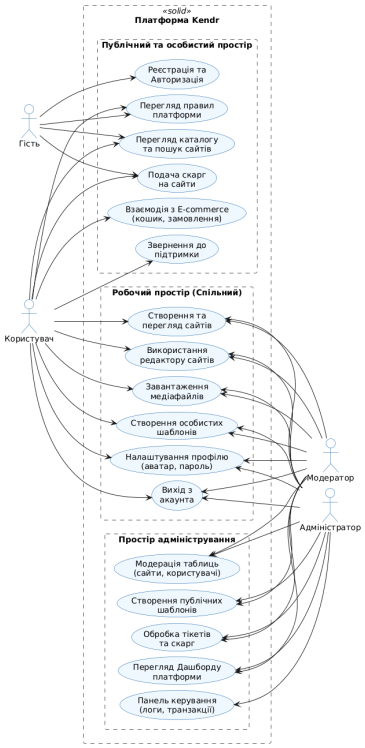

Рис. 2.2 – Діаграма варіантів використання системи

Візуалізація чітко розмежовує зони відповідальності: актори базового рівня (Гість, Користувач) взаємодіють переважно з публічною частиною та робочим простором, тоді як службові ролі (Модератор, Адміністратор) мають доступ до адміністративних інструментів модерації та контролю. Розроблена модель слугує надійним фундаментом для подальшого проєктування об'єктної структури класів та бази даних.

Для більш детального розуння логіки роботи системи та підготовки до проєктування алгоритмів, було розроблено специфікації для ключових прецедентів платформи. Специфікація деталізує кроки взаємодії актора з системою, визначає передумови та можливі альтернативні сценарії (обробку помилок). Це дозволяє чітко сформулювати контракт між клієнтською частиною та сервером.

У таблиці 2.2 наведено специфікацію базового варіанта використання "Реєстрація та Авторизація", яка забезпечує безпечний доступ до робочого простору та управління сесіями користувачів.

Таблиця 2.2 – Специфікація прецеденту "Реєстрація та Авторизація"

Атрибут	Опис
Назва прецеденту	Реєстрація та Авторизація
Актор	Гість
Мета	Отримати доступ до закритого функціоналу створення, редагування сайтів та управління профілем.
Передумови	Гість знаходиться на публічній сторінці платформи, має стабільне інтернет-з'єднання.
Основний сценарій (Успішний)	1. Гість обирає опцію "Увійти" на навігаційній панелі. 2. Система відображає форму для вводу електронної пошти та пароля. 3. Гість вводить свої облікові дані та натискає "Увійти". 4. Клієнтський застосунок валідує формат введених даних. 5. Сервер перевіряє наявність користувача в БД (або створює нового). 6. Сервер генерує криптографічний JWT-токен доступу та повертає його клієнту. 7. Система зберігає токен і перенаправляє користувача до ізольованого Робочого простору.
Альтернативний сценарій (Помилки)	4а. Введено некоректний email або слабкий пароль: система блокує відправку запиту, виводить повідомлення про помилку валідації та просить повторити ввід. 5а. (При реєстрації) Користувач з таким email вже існує: сервер повертає статус 409 Conflict, виводить повідомлення про конфлікт. 5б. (При вході) Невірний пароль або email: сервер повертає статус 401 Unauthorized, виводиться повідомлення про помилку без уточнення на конкретне поле.
Постумови	Гість набуває ролі "Користувач". У системі створюється активна авторизована сесія.

Основним інноваційним функціоналом платформи є маніпуляція DOM-деревом через візуальний інтерфейс. У таблиці 2.3 описано інженерну специфікацію прецеденту "Використання редактору сайтів", який є найскладнішим компонентом клієнтської частини.

Таблиця 2.3 – Специфікація прецеденту "Використання редактору сайтів"

Атрибут	Опис
Назва прецеденту	Використання редактору сайтів
Актор	Користувач
Мета	Створити або змінити структуру та зовнішній вигляд власного вебсайту без безпосереднього написання вихідного коду.
Передумови	Користувач авторизований у системі та відкрив існуючий або ініціалізував новий проєкт.
Основний сценарій (Успішний)	1. Користувач відкриває маршрут візуального редактора. 2. Система виконує HTTP GET запит, завантажує поточну JSON-конфігурацію сайту з бази даних та рендерить ієрархію компонентів на екрані. 3. Користувач обирає необхідний функціональний блок (наприклад, "Кнопка" або "Текстовий блок") з бічної панелі інструментів. 4. За допомогою Drag-and-Drop користувач перетягує блок у робочу область; система розраховує місце вставки.

Продовження таблиці 2.3

	5. Користувач виділяє доданий блок та змінює його параметри (колір, типографіку, відступи) через панель налаштувань. 6. Користувач натискає кнопку "Зберегти". 7. Система серіалізує поточний стан віртуального DOM у формат JSON і відправляє PUT запит на Backend API. 8. Сервер валідує схему даних і успішно оновлює відповідний запис у базі даних.
Альтернативний сценарій (Помилки)	7а. Втрата з'єднання з інтернетом: система перехоплює помилку мережі, сповіщає користувача про неможливість синхронізації та тимчасово кешує зміни локально у LocalStorage браузера. 8а. Помилка валідації JSON на сервері (порушення структури): сервер відхиляє запит (статус 400 Bad Request), система повідомляє про критичну помилку стану та пропонує перезавантажити сторінку до останньої стабільної версії.
Постумови	Оновлена JSON-структура ресурсу надійно збережена в базі даних та готова до компіляції або публікації.

Окрім клієнтського функціоналу, стабільність платформи підтримується адміністративними процесами. Специфікація прецеденту модерації (табл.2.4) демонструє захисні механізми системи та зміну глобальних станів об'єктів.

Таблиця 2.4 – Специфікація прецеденту "Модерація таблиць "

Атрибут	Опис
Назва прецеденту	Модерація таблиць (Блокування сайту)
Актор	Модератор, Адміністратор
Мета	Обмежити публічний доступ до вебресурсу, що порушує контентну політику платформи (спам, шахрайство).
Передумови	Модератор авторизований у системі, знаходиться в адміністративній панелі керування та ідентифікував сайт-порушник.
Основний сценарій (Успішний)	1. Модератор знаходить необхідний сайт через систему фільтрації у таблиці всіх проєктів. 2. Модератор відкриває картку проєкту і натискає дію "Призупинити сайт". 3. Система вимагає обов'язкового введення причини блокування. 4. Модератор вводить причину та підтверджує дію. 5. Сервер виконує оновлення запису в базі даних, змінюючи статус сайту з published на suspended. 6. Сервер автоматично генерує об'єкт "Попередження", прив'язуючи його до власника сайту. 7. Система відправляє автоматичний електронний лист власнику сайту із зазначенням причини блокування та інструкцією для подачі апеляції.
Альтернативний сценарій (Помилки)	5а. Конкурентний доступ (Сайт вже видалено власником): сервер не знаходить активного запису в базі (статус 404 Not Found), система виводить помилку модератору та оновлює таблицю. 5б. Спроба модератора заблокувати сайт іншого модератора/адміністратора: система перевіряє рівні доступу та відхиляє операцію (статус 403 Forbidden).
Постумови	Маршрутизатор платформи починає віддавати HTTP-статус 403 (доступ заборонено) при спробі модератора видалити ресурс іншого модератора або адміністратора.

Розроблені специфікації прецедентів визначають архітектурні контракти системи. Вони демонструють, що кожна дія актора супроводжується серверною логікою: управлінням транзакціями, валідацією прав доступу, обробкою помилок мережі та синхронізацією розподілених баз даних. Ця деталізація є вхідними даними для наступного етапу – проєктування об'єктно-орієнтованої структури класів та маршрутів API.

2.2 Проєктування логічної структури бази даних

Обгрунтування вибору СУБД та гібридного підходу

Для забезпечення надійного зберігання інформації, підтримки транзакційної цілісності даних було обрано реляційну систему управління базами даних MySQL. Ця система гарантує виконання вимог ACID (атомарність, узгодженість, ізольованість, довговічність), що є критично важливим для модулів електронної комерції та білінгу.

Однак, враховуючи специфіку NoCode платформи, де структура згенерованого сайту є максимально динамічною і не має фіксованого набору полів (кожна сторінка може мати унікальний набір блоків), для зберігання контенту сторінок та налаштувань візуальних тем було використано нативний тип даних JSON, який підтримується в сучасних версіях MySQL. Цей архітектурний крок дозволяє ідеально поєднати надійність строгих SQL-зв'язків для системних сутностей із гнучкістю NoSQL-документів для користувацького контенту.

Логічне проєктування

Логічна структура бази даних платформи складається з понад 20 взаємопов'язаних таблиць, які покривають управління користувачами, створення сайтів, систему електронної комерції, медіатеку та адміністративні функції.

Для уникнення надлишковості інформації та надійної підтримки цілісності даних, більшість таблиць реляційної моделі було нормалізовано до 3НФ. Наведений нижче графічний концепт у вигляді ER-діаграми детально ілюструє всі ключові сутності, їхні атрибути, ключі, а також типи логічних зв'язків між ними, що формують єдиний інформаційний простір платформи.

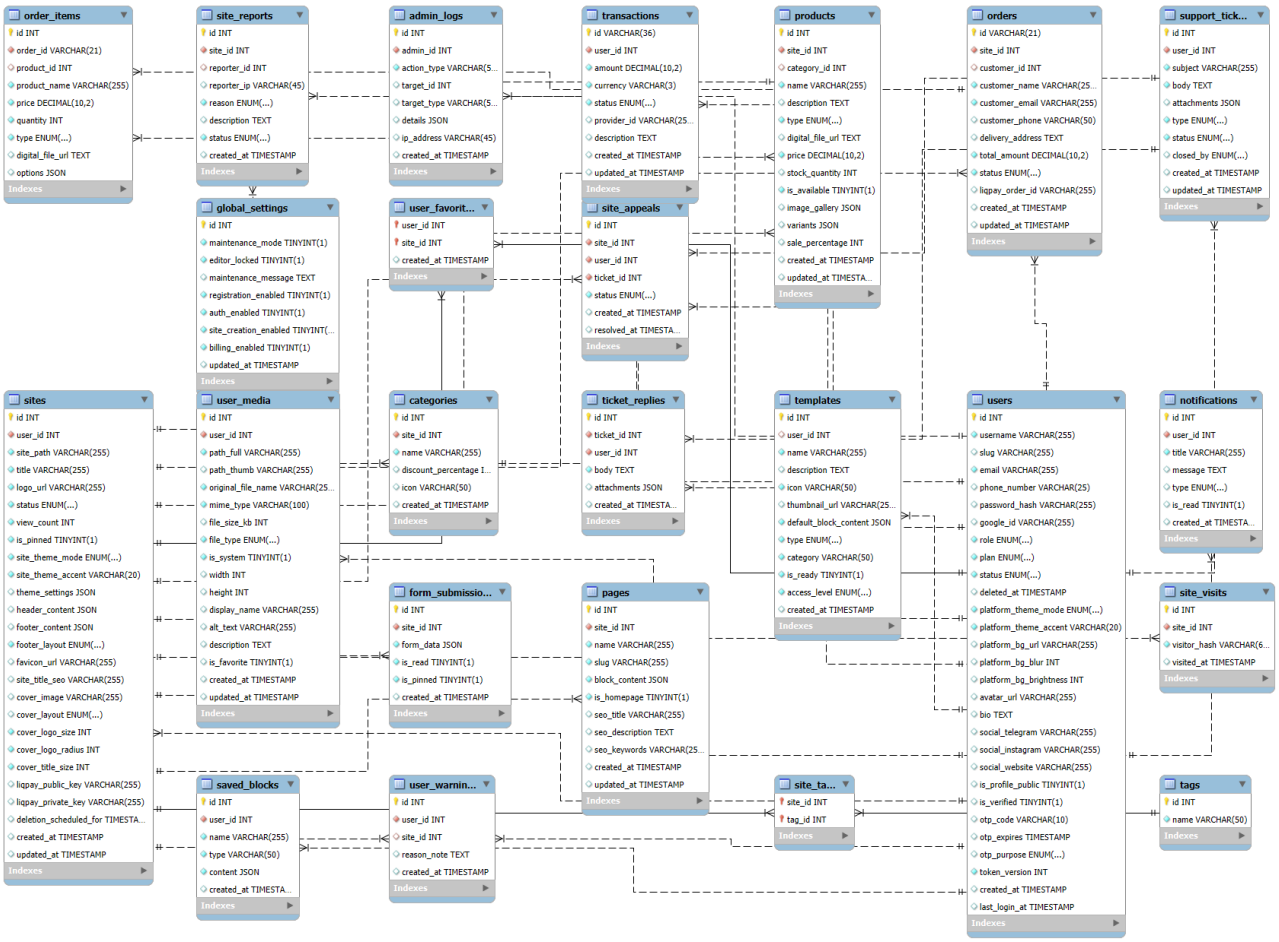

Рис. 2.3 – ER-діаграма реляційної бази даних KendrDB

Ключові зв'язки між сутностями системи побудовані за такими правилами:

- Між сутністю users та sites встановлено зв'язок один-до-багатьох (1:N). Один користувач може створити декілька проєктів, але кожен проєкт належить лише одному власнику. Видалення користувача призводить до каскадного видалення всіх його сайтів.
- Сутність sites пов'язана із сутністю pages зв'язком 1:N (один сайт має багато сторінок). Комбінація ідентифікатора сайту та шляху сторінки (site_id, slug) є унікальною.
- Реалізовано зв'язок багато-до-багатьох (M:N) між таблицями sites та tags через проміжну таблицю site_tags, що дозволяє фільтрувати сайти в публічному каталозі за кількома тегами одночасно.
- Модуль магазину пов'язує таблицю orders з таблицею products через транзитну таблицю order_items, що дозволяє зберігати історію цін товарів на момент покупки.

- **Фізична структура БД**
- Для реалізації логічної моделі було створено фізичні таблиці. У таблицях 2.5 – 2.8 наведено специфікації найважливіших сутностей системи, що формують ядро платформи.
- Для безпечного зберігання облікових записів, налаштувань підписки та UI-преференцій розроблено таблицю users. Її фізичну структуру наведено в таблиці 2.7.

Таблиця 2.5 – Структура таблиці users

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор користувача.
username	VARCHAR(255)	UNIQUE, NOT NULL	Унікальне ім'я користувача в системі.
email	VARCHAR(255)	UNIQUE, NOT NULL	Електронна пошта.
password_hash	VARCHAR(255)	NULL	Хешований пароль (bcrypt). Може бути NULL при OAuth 2.0.
role	ENUM	NOT NULL, DEFAULT 'user'	Роль у системі ('user', 'moderator', 'admin').
plan	ENUM	NOT NULL, DEFAULT 'FREE'	Тарифний план ('FREE', 'PLUS', 'ADMIN').
status	ENUM	NOT NULL, DEFAULT 'active'	Стан акаунту ('active', 'suspended', 'deleted').
is_verified	BOOLEAN	DEFAULT FALSE	Прапорець підтвердження електронної пошти.
platform_theme_mode	ENUM	NOT NULL, DEFAULT 'system'	Налаштування теми інтерфейсу.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата та час реєстрації облікового запису.

Основною сутністю генератора є вебсайт. Таблиця sites (таблиця 2.6) має загальні налаштування проєкту, параметри SEO та ключі для прийому платежів.

Таблиця 2.6 – Структура таблиці sites

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор вебсайту.
user_id	INT	FK, NOT NULL	Посилання на власника сайту.
site_path	VARCHAR(255)	UNIQUE, NOT NULL	Унікальний субдомен або шлях ресурсу.
title	VARCHAR(255)	NOT NULL	Назва проєкту.
status	ENUM	NOT NULL, DEFAULT 'private'	Поточний статус сайту.
theme_settings	JSON	NULL	Глобальні налаштування візуальної теми сайту.
liqpay_public_key	VARCHAR(255)	NULL	Публічний ключ LiqPay.

Для збереження безпосереднього контенту, який користувач створює через Drag-and-Drop редактор, використовується таблиця pages (таблиця 2.7).

Таблиця 2.7 – Структура таблиці pages

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор сторінки.
site_id	INT	FK, NOT NULL	Посилання на сайт, якому належить сторінка.
name	VARCHAR (255)	NOT NULL	Системна назва сторінки.
slug	VARCHAR (255)	NOT NULL	URL-шлях сторінки (наприклад, "/about").
block_content	JSON	NOT NULL	Масив компонентів сторінки, згенерованих у редакторі.
is_homepage	BOOLEAN	NOT NULL, DEFAULT 0	Прапорець, що визначає сторінку як головну.
seo_title	VARCHAR (255)	NULL	Заголовок для пошукової оптимізації.

Для забезпечення якісного обслуговування користувачів та модерації реалізовано підсистему тікетів. Структуру таблиці support_tickets наведено у таблиці 2.8.

Таблиця 2.8 – Структура таблиці support_tickets

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор звернення.
user_id	INT	FK, NOT NULL	Ідентифікатор користувача, який створив тікет.
subject	VARCHAR(255)	NOT NULL	Тема звернення.
body	TEXT	NOT NULL	Текст проблеми або питання.
attachments	JSON	NULL	Масив посилань на прикріплені файли/скріншоти.
status	ENUM	NOT NULL, DEFAULT 'open'	Стан тікета.
closed_by	ENUM	NULL	Вказує, хто закрив тікет.

Опис архітектури JSON-схеми сайту

Ключовим інноваційним та архітектурним рішенням платформи є спосіб збереження сторінок користувача. Замість створення десятків таблиць для кожного можливого елемента інтерфейсу, структура поля block_content у таблиці pages використовує формат JSON.

Дані зберігаються у вигляді впорядкованого масиву JavaScript-об'єктів, де кожен об'єкт представляє окремий функціональний блок на сторінці.

Стандартний вузол має сувору схему і містить такі ключі:

- id – унікальний ідентифікатор блоку, що необхідний для коректної роботи алгоритмів рендерингу React та переміщення блоків.
- type – строкова константа, що визначає тип компонента для фабрики рендерингу.
- props – вкладений об'єкт із конфігурацією конкретного блоку. Включає текстовий контент, URL-посилання на медіафайли та параметри стилізації.

Приклад фрагмента JSON-схеми, що зберігається в базі даних:

```
[
  {
    "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "type": "Hero",
    "props": {
      "title": "Ласкаво просимо до мого портфоліо",
      "subtitle": "Вебдизайнер та розробник",
      "layout": "centered",
      "backgroundColor": "#1a1a1a"
    }
  },
  {
    "id": "c9bf9e57-1685-4c89-bafb-ff5af830be8a",
    "type": "Text",
    "props": {
      "content": "Тут ви знайдете мої найкращі роботи...",
      "fontSize": 16,
      "alignment": "left"
    }
  }
]
```

Такий підхід забезпечує максимальну продуктивність: під час завантаження сайту Backend виконує лише один SQL-запит до таблиці pages, отримує готовий JSON-масив і відправляє його на клієнт, де React миттєво відмальовує весь інтерфейс без необхідності додаткових обчислень на сервері (реалізація архітектури One-way data flow).

2.3 Проєктування компонентної структури та взаємодії модулів

Діаграма розгортання призначена для моделювання фізичної та інфраструктурної архітектури системи. За допомогою цієї діаграми можна

оцінити конфігурацію серверних потужностей та хмарних сервісів, на яких працюватимуть окремі компоненти платформи, а також описати топологію їхньої взаємодії.

Оскільки система проєктується як сучасне SaaS-рішення, її архітектура відходить від традиційного монолітного розміщення на одному фізичному сервері на користь розподіленої хмарної інфраструктури. На рис. 2.4 представлена розроблена діаграма розгортання програмного комплексу.

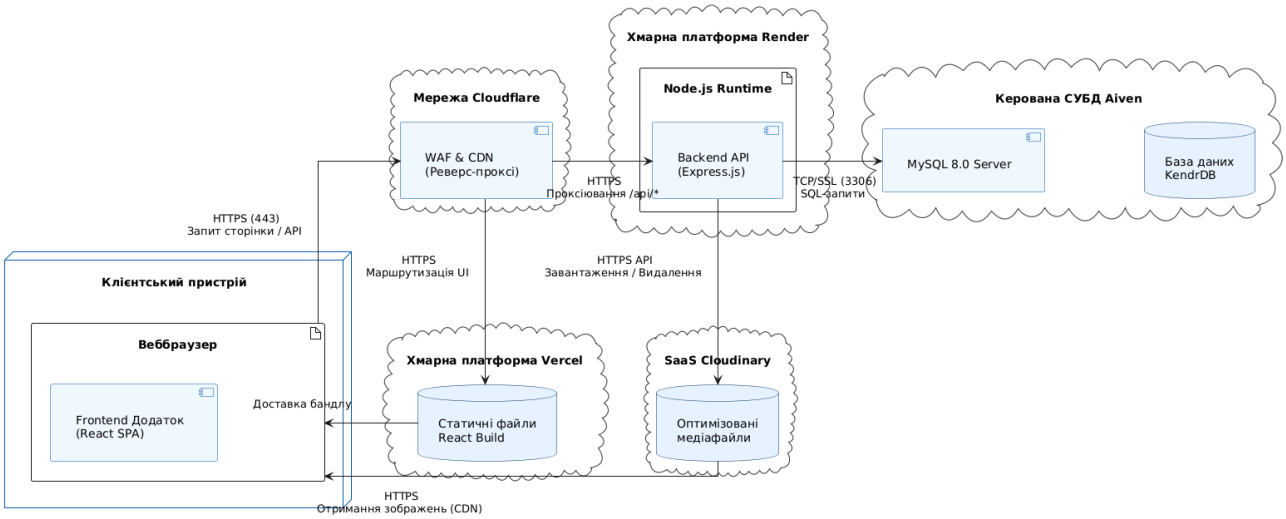

Рис. 2.4 – Діаграма розгортання системи

Архітектура платформи складається з таких ключових інфраструктурних вузлів:

Клієнтський пристрій: Фізичний пристрій кінцевого користувача, на якому запущено сучасний веббраузер. На цьому вузлі виконується завантажений артефакт клієнтського застосунку (React SPA), який відповідає за рендеринг візуального редактора, управління локальним станом та реакцію на дії користувача.

Мережа Cloudflare: Виступає єдиною точкою входу для всіх клієнтських запитів. Cloudflare функціонує як глобальний реверс-проксі сервер, забезпечуючи захист від DDoS-атак, фільтрацію зловмисного трафіку та управління SSL-сертифікатами. Цей вузол виконує маршрутизацію: запити на отримання інтерфейсу спрямовуються на Vercel, а API-запити на Render.

Хмарна платформа Vercel (Frontend Server): Вузол, оптимізований для хостингу статичних файлів та фреймворків. Тут зберігається скомпільований "React Build" (HTML, CSS, JavaScript-бандли). Vercel забезпечує миттєву доставку інтерфейсу користувачу через власну глобальну мережу доставки контенту.

Хмарна платформа Render (Application Server): Вузол, на якому виконується основна бізнес-логіка системи. Тут розгорнуто середовище виконання Node.js, всередині якого працює Backend API на базі Express.js. Сервер обробляє авторизацію, валідацію даних, логіку модерації та генерацію конфігурацій сайтів. Завдяки Render, цей вузол автоматично масштабується при зростанні навантаження.

Керована база даних Aiven: Ізольований вузол для зберігання структурованих даних, що працює за моделлю DBaaS. На ньому розгорнуто MySQL 8.0 Server, який обслуговує базу даних KendrDB. Доступ до цього вузла обмежений: підключитися до бази даних може виключно сервер застосунків Render.

Медіасервіс Cloudinary: Спеціалізоване хмарне сховище для роботи із зображеннями. Воно повністю замінює локальну файлову систему для збереження користувацьких аватарів, логотипів та контенту сайтів.

Архітектурна взаємодія компонентів системи

Сценарій взаємодії в системі ініціюється Клієнтським пристроєм. Коли користувач вводить адресу платформи, запит проходить через фільтри безпеки Cloudflare і маршрутизується до Vercel, який повертає клієнту зібраний React-додаток.

Після завантаження інтерфейсу у браузері, React-додаток починає взаємодіяти із сервером для отримання або збереження даних. Він надсилає HTTP-запити за маршрутом /api/.... Cloudflare розпізнає ці API-запити і безпечно проксіює їх на Render.

Node.js-сервер приймає запит, виконує необхідну бізнес-логіку і надсилає SQL-запит до бази даних Aiven. Після отримання результатів від БД, сервер формує JSON-відповідь і повертає її клієнту.

Робота з файлами має окремий потік. Під час завантаження користувачем зображення, Backend відправляє файл до Cloudinary через захищене API, отримуючи натомість оптимізоване посилання. Надалі, під час перегляду створених сайтів, клієнтські браузери завантажують зображення напряму з глобальної CDN Cloudinary, повністю минаючи основний сервер платформи.

Така мікросервісна та хмарна архітектура гарантує високу відмовостійкість системи, швидку доставку контенту користувачам у будь-якій точці світу та безпеку зберігання даних.

Програмна реалізація системи базується на принципах модульності, слабкої зв'язності та суворого розділення відповідальності. Для проєктування структури програмного коду було використано методології об'єктно-орієнтованого та компонентного програмування, а також класичні патерни проєктування .

Об’єктно-орієнтована доменна модель системи

Доменна модель описує ключові сутності бізнес-логіки та зв'язки між ними, абстрагуючись від конкретної реалізації у програмному коді. Далі (рис.2.5) подано діаграму класів предметної області без урахування архітектурних патернів.

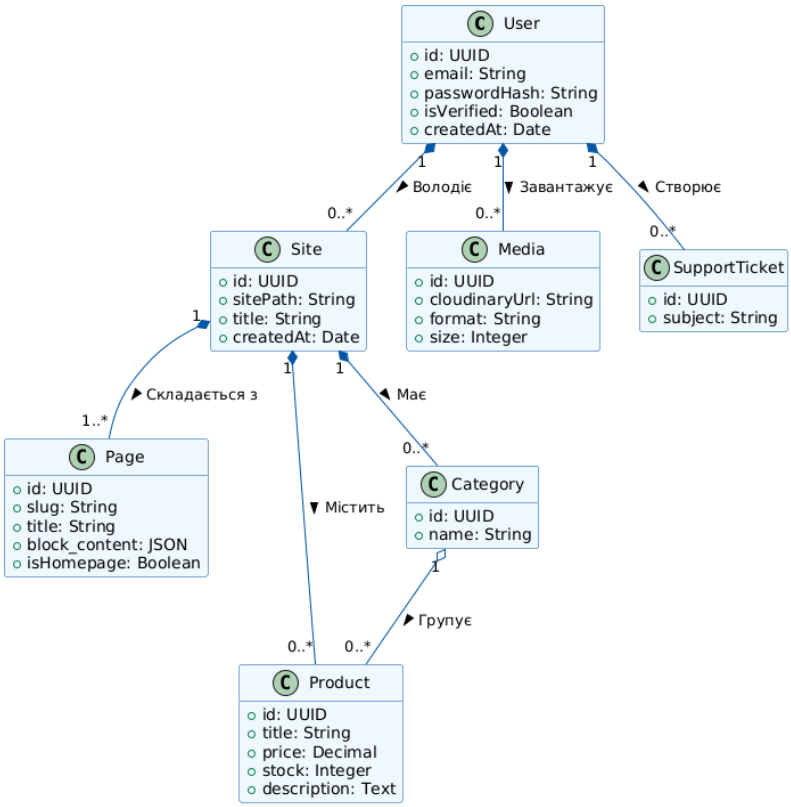

Рис. 2.5 – Діаграма класів предметної області

Основною сутністю предметної області є клас User, який представляє обліковий запис. Він має відношення композиції до класів Site, Media та SupportTicket, оскільки ці об'єкти фізично не можуть існувати без власника-користувача. Клас Site є агрегатором для контенту. Він пов'язаний суворою композицією з класом Page, де ключовим атрибутом є block_content типу JSON – саме в ньому зберігається вся структура візуального редактора. Додатково до сайту можуть належати об'єкти класів Product та Category, які формують підсистему електронної комерції.

Патерни проєктування та архітектура реалізації

Для забезпечення масштабованості коду та уникнення його дублювання, розробка спирається на використання сталих патернів проєктування. Далі (рис.2.6) зображено діаграму класів реалізації, яка ілюструє проходження потоку даних від клієнтського застосунку до бази даних.

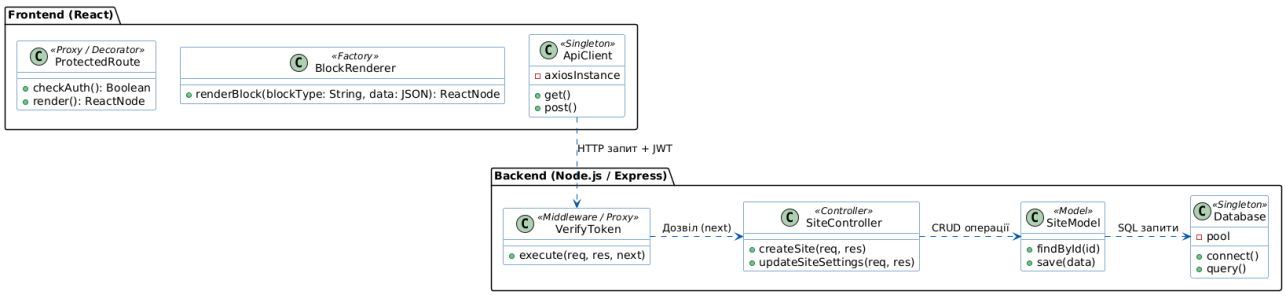

Рис. 2.6 – Діаграма класів реалізації з використанням патернів

У системі реалізовано такі архітектурні патерни:

MVC (Model-View-Controller): Фундаментальний патерн серверної частини. Маршрутизатор делегує запит Контролеру (наприклад, SiteController), який містить бізнес-логіку та звертається до Моделі (SiteModel) для взаємодії з базою даних. Представленням (View) у цій архітектурі виступає відокремлений React-застосунок.

Singleton: Застосовується для глобальних конфігурацій. На клієнті це екземпляр ApiClient (налаштований клієнт axios), а на сервері – клас Database, пул з'єднань із MySQL у файлі db.js, що запобігає створенню надлишкових підключень до бази даних.

Factory Method: Використовується в ядрі візуального редактора. Компонент BlockRenderer отримує тип блоку (наприклад, "Hero" або "Text") з JSON-схеми і динамічно генерує та повертає відповідний React-компонент для відмальовування.

Proxy / Decorator (Замісник / Декоратор): На сервері цей патерн реалізовано через Middleware (наприклад, verifyToken), які перехоплюють запит для перевірки JWT-токена перед допуском до Контролера. На клієнті цю роль виконують Higher-Order Components, такі як ProtectedRoute, що блокують доступ до закритих маршрутів неавторизованим користувачам.

Компонентна структура клієнтської та серверної частин

Фізична структура програмного коду поділена на два ізольовані застосунки (монорепозиторій), взаємодія між якими відбувається виключно через REST API. Діаграму компонентів, що відображає файлову та модульну організацію проєкту, наведено нижче (рис. 2.7).

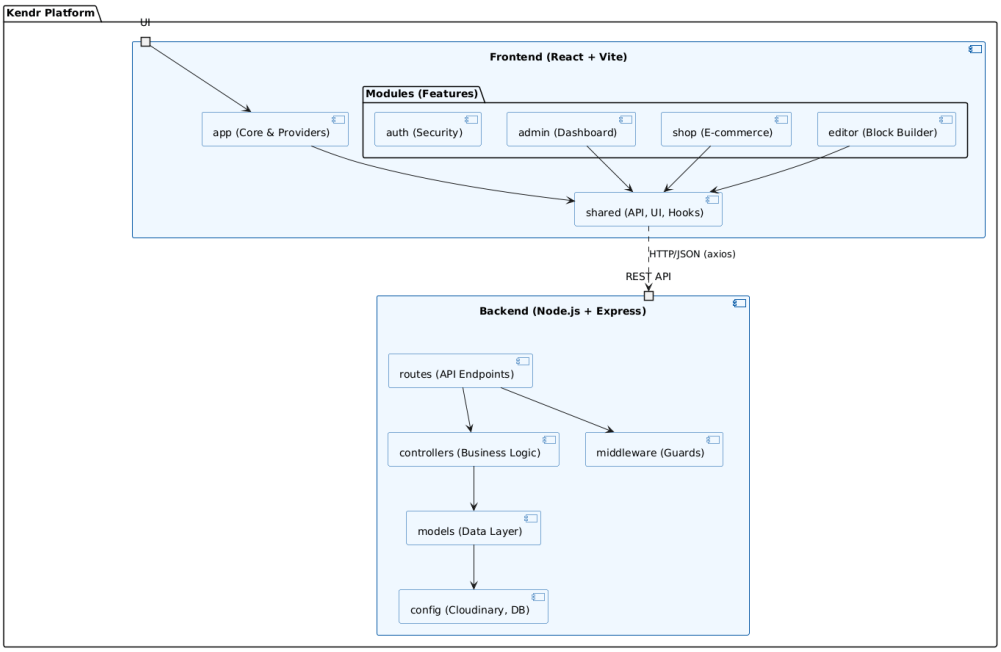

Рис. 2.7 – Діаграма компонентів програмного комплексу

Серверна частина (Node.js / Express) побудована за модульним принципом. Папка routes відповідає за декларацію кінцевих точок API, направляючи трафік до controllers. Безпека забезпечується шаром middleware, який містить захисні екрани

(verifyAdmin, verifyTurnstile). Зв'язок із зовнішніми сервісами винесено у папку config (конфігурація Cloudinary та підключення до бази даних).

Клієнтська частина (React + Vite) спроєктована за принципом Feature-Sliced Design (модульна архітектура). Код поділено на три великі блоки:

app: Ядро застосунку, що містить глобальні провайдери контексту та налаштування маршрутизації.

modules: Незалежні фіче-модулі платформи. Найбільшим є модуль editor, який інкапсулює логіку Drag-and-Drop, панелі налаштувань та бібліотеку блоків. Також тут розміщені модулі admin (панель керування), shop (магазин) та dashboard (робочий простір користувача).

shared: Спільний простір, що містить універсальні UI-елементи (кнопки, модальні вікна), утиліти для генерації коду (htmlGenerator.js) та кастомні хуки (useAutoSave).

Така чітка декомпозиція дозволяє незалежно тестувати, оновлювати та масштабувати окремі частини платформи без ризику пошкодити загальну бізнес-логіку системи. Повну структуру каталогів та файлів розробленого програмного забезпечення, що детально відображає фізичну організацію монорепозиторію, наведено у Додатку В.

Проєктування взаємодії компонентів та програмних інтерфейсів

Для деталізації процесів обміну даними між клієнтською та серверною частинами платформи розроблено діаграми взаємодії. Вони ілюструють життєвий цикл обробки HTTP-запитів в умовах REST API архітектури, передачу управління між контролерами та безпосередні звернення до бази даних.

Діаграми послідовності

Діаграми послідовності дозволяють відобразити хронологічний порядок виклику методів та передачі повідомлень між об'єктами (екземплярами класів) під час виконання конкретного сценарію.

На рисунку 2.8 зображено діаграму послідовності процесу створення нового вебсайту, який є базовим функціоналом платформи.

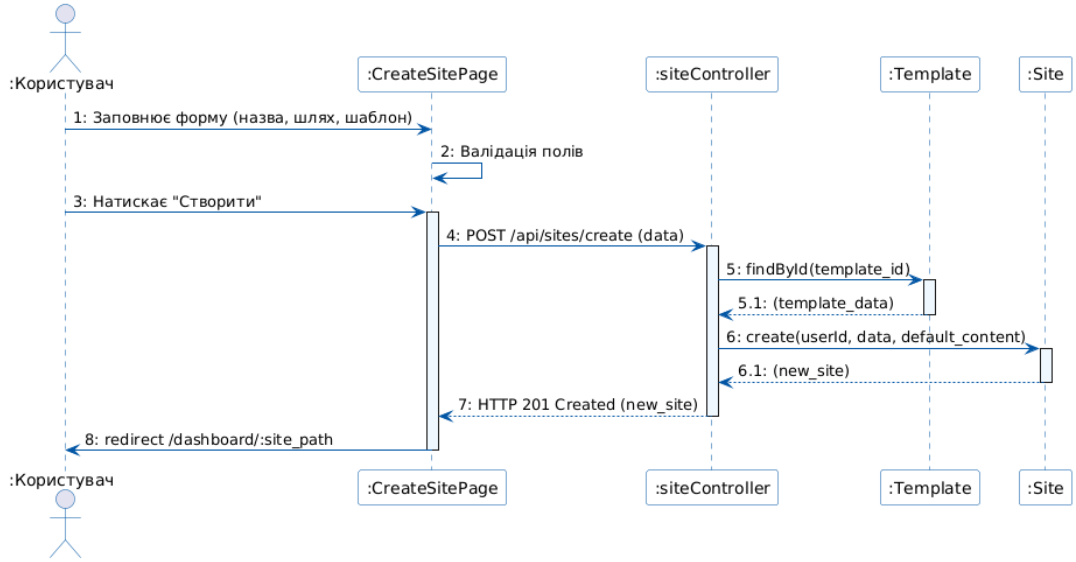

Рис. 2.8 – Діаграма послідовності "Створення сайту"

Взаємодія між об'єктами під час створення проєкту відбувається в такій хронології:

a. Актор (Користувач) взаємодіє з клієнтським інтерфейсом CreateSitePage, заповнюючи форму базовими даними.
b. Компонент CreateSitePage виконує локальну синхронну валідацію полів.
c. Користувач ініціює відправку даних, натискаючи кнопку "Створити".
d. Клієнт формує асинхронний HTTP POST запит на маршрут /api/sites/create і передає JSON з даними. Запит обробляється маршрутизатором та делегується siteController.
e. siteController звертається до моделі TemplateModel для отримання базового JSON-контенту обраного шаблону.
f. Отримавши контент шаблону, контролер викликає метод create() моделі SiteModel, передаючи ідентифікатор користувача, дані форми та дефолтний контент.
g. Після успішного виконання SQL-запиту, контролер формує відповідь зі статусом 201 Created і повертає об'єкт нового сайту клієнту.
h. Клієнтський застосунок успішно завершує транзакцію та виконує перенаправлення користувача до робочого простору створеного сайту.

На рисунку 2.9 подано діаграму послідовності створення звернення до служби підтримки.

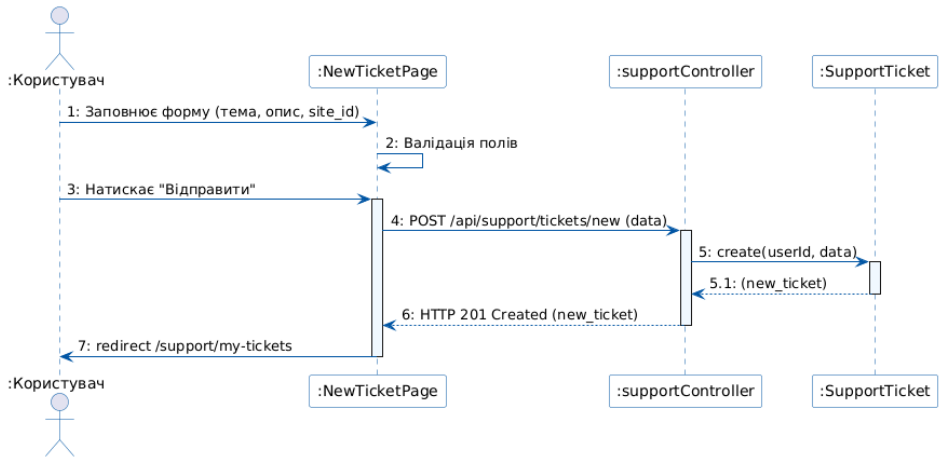

Рис. 2.9 – Діаграма послідовності "Створення тікету підтримки"

Логіка обміну даними при формуванні тікету підтримки:

a. Актор (Користувач) вводить деталі проблеми у компоненті NewTicketPage.
b. Інтерфейс здійснює локальну перевірку наявності обов'язкових полів.
c. Користувач підтверджує відправку форми.
d. Клієнт ініціює HTTP POST запит на /api/support/tickets/new, передаючи корисне навантаження до supportController.
e. Контролер звертається до моделі SupportTicket, викликаючи метод create().
f. Після успішного збереження запису в базу даних, контролер повертає статус 201 Created.
g. Клієнтський маршрутизатор перенаправляє користувача на сторінку зі списком активних тікетів.

На рисунку 2.10 ілюструється складна бізнес-транзакція – процес призупинення сайту адміністратором платформи, який містить розгалуження та умовні виклики.

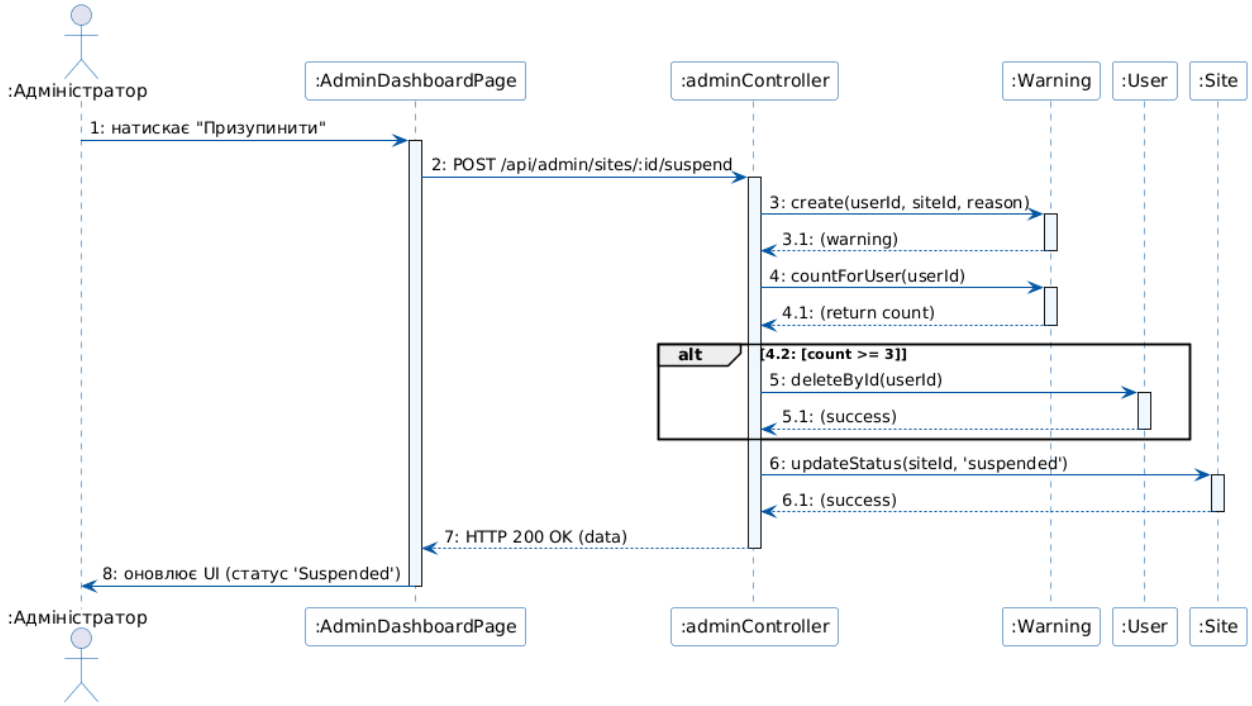

Рис. 2.10 – Діаграма послідовності "Призупинення сайту"

Хронологія адміністративної модерації:

a. Актор (Адміністратор) ініціює блокування сайту-порушника через компонент AdminDashboardPage.
b. Відправляється HTTP POST запит на захищений маршрут /api/admin/sites/:id/suspend до adminController.
c. Контролер першочергово звертається до WarningModel (метод create()) для фіксації попередження в профілі порушника.
d. Контролер виконує повторний запит до WarningModel (countForUser()), щоб отримати загальну кількість активних попереджень користувача.
e. Відбувається умовне розгалуження (alt): якщо кількість попереджень досягає критичної межі (count ≥ 3), контролер звертається до UserModel та ініціює каскадне видалення облікового запису (deleteById()).
f. Незалежно від попереднього кроку, контролер звертається до SiteModel та виконує оновлення статусу ресурсу (updateStatus()).
g. Контролер повертає успішний статус 200 OK клієнту.
h. Інтерфейс адміністратора синхронізується з сервером, візуально позначаючи сайт статусом "Suspended".

Діаграма кооперації

Для глибшого розуміння структурних зв'язків між об'єктами під час складних транзакцій використовується діаграма кооперації. Вона відображає ті самі повідомлення, що й діаграма послідовності, але акцентує увагу не на часі, а на просторовій архітектурі та топології зв'язків.

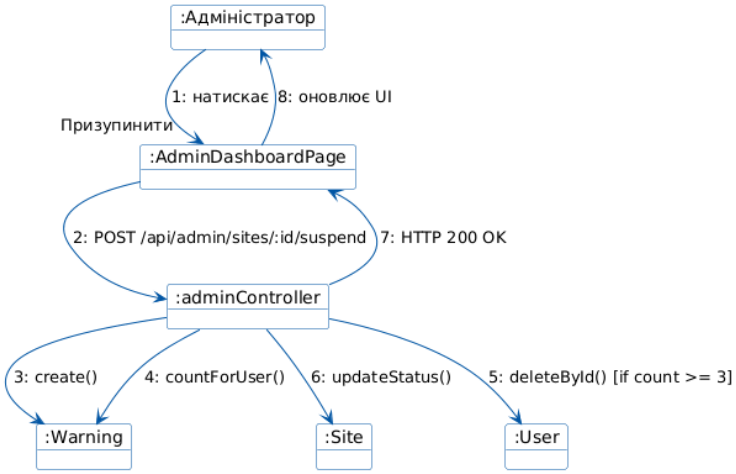

Рис. 2.11 – Діаграма кооперації "Призупинення сайту"

Діаграма демонструє радіальну топологію управління: adminController виступає центральним вузлом, який ізолює клієнтський інтерфейс (AdminDashboardPage) від безпосередньої взаємодії з шаром даних. Контролер бере на себе відповідальність за координацію трьох різних моделей (Warning, User, Site), забезпечуючи виконання принципу єдиної відповідальності та транзакційної цілісності операції модерації.

2.4 Розробка основних алгоритмів роботи платформи

Для детального розуміння внутрішньої логіки платформи розроблено алгоритми роботи ключових бізнес-процесів. Візуалізація алгоритмів здійснена за допомогою діаграм активності. Ці діаграми відображають динаміку поведінки системи, показуючи паралельні процеси, розгалуження та чіткий розподіл зон відповідальності між Клієнтом та Сервером за допомогою доріжок.

Алгоритм процесу "Створення сайту користувачем"

Цей алгоритм описує один із найважливіших процесів платформи – ініціалізацію нового вебресурсу на базі готового шаблону.

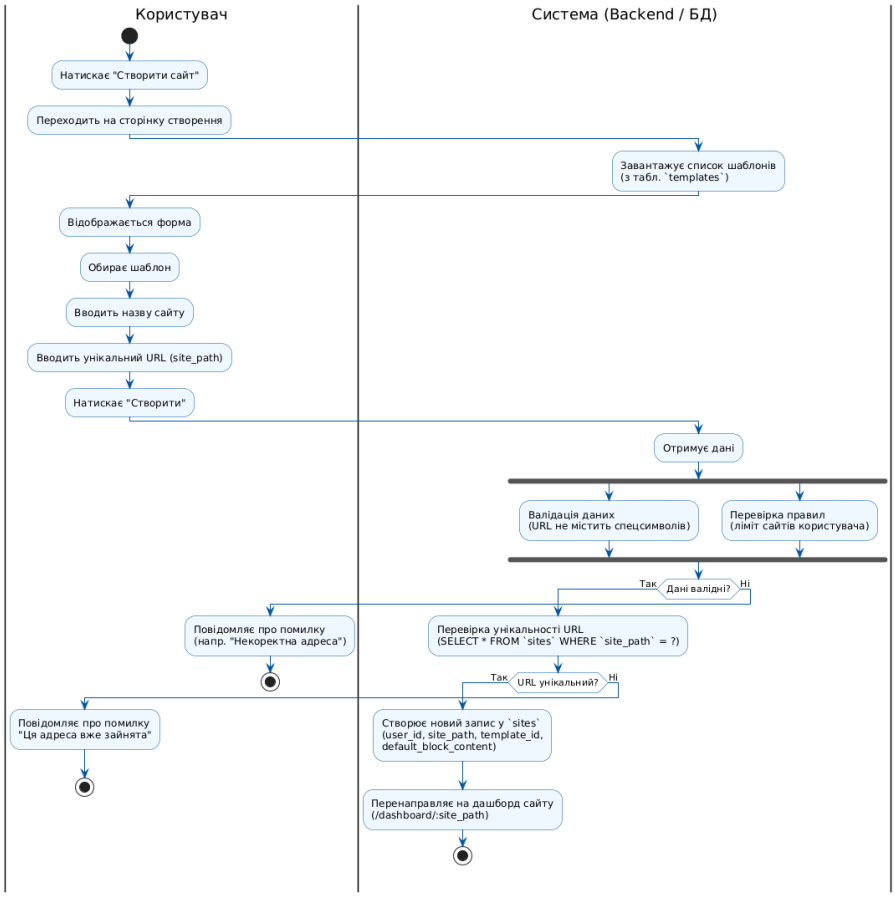

Рис. 2.12 – Діаграма активності "Створення сайту"

Покроковий опис алгоритму:

a. **Ініціалізація:** Користувач натискає кнопку створення сайту. Система звертається до бази даних і завантажує масив доступних публічних та системних шаблонів (таблиця templates).

b. **Введення даних:** Користувач обирає шаблон і заповнює форму.

c. **Паралельна валідація:** Backend одночасно виконує дві дії: перевіряє правильність введених даних та перевіряє бізнес-правила.

d. **Перевірка унікальності:** Якщо дані валідні, система робить SQL-запит до таблиці sites, перевіряючи, чи вільний обраний site_path.

e. **Завершення:** У разі успіху створюється запис у БД з копіюванням default_block_content із шаблону, а користувача перенаправляють у робочий простір.

Альтернативні гілки: Якщо дані невалідні або URL вже зайнятий, алгоритм переривається, а Frontend виводить відповідне повідомлення про помилку.

Алгоритм процесу "Редагування контенту (Блоковий редактор)"

Цей процес демонструє оптимізовану роботу з візуальним конструктором, де основне навантаження лягає на клієнтську частину, а звернення до сервера мінімізовані.

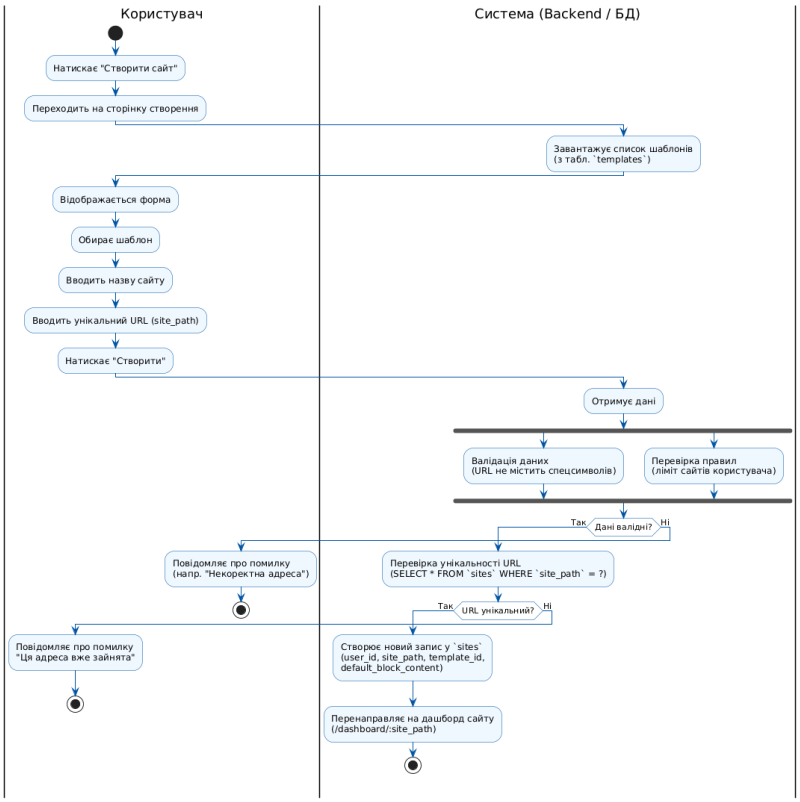

Рис. 2.13 – Діаграма активності "Редагування контенту"

Покроковий опис алгоритму:

a. **Завантаження стану:** При відкритті редактора Backend передає на клієнт поточний масив блоків з таблиці pages.

b. **Рендеринг:** Frontend відмальовує інтерфейс віртуального DOM.

c. **Взаємодія:** Користувач маніпулює блоками виключно в локальній пам'яті браузера. Він може паралельно додавати, редагувати, видаляти блоки або змінювати їх порядок за допомогою Drag-and-Drop. На цьому етапі запити до бази даних не виконуються.

d. **Синхронізація:** Після натискання кнопки "Зберегти" Frontend формує фінальний оновлений JSON.

e. **Збереження:** Backend приймає JSON, виконує SQL-запит UPDATE та повертає статус 200 OK, після чого інтерфейс сповіщає про успішне збереження.

Алгоритм процесу "Взаємодія клієнта з сайтом-магазином"

Алгоритм описує логіку додавання товару до кошика на згенерованому сайті-магазині, враховуючи перевірку залишків та авторизації.

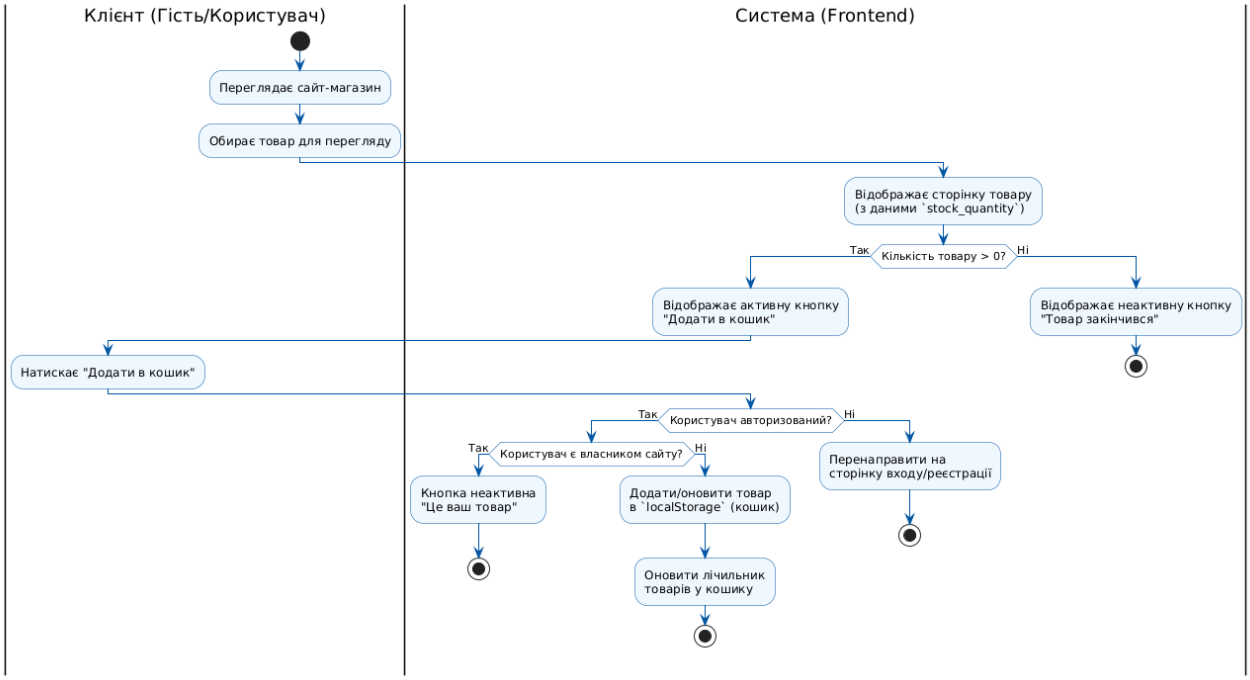

Рис. 2.14 – Діаграма активності "Взаємодія клієнта з магазином"

Покроковий опис алгоритму:

a. **Перегляд:** Клієнт відкриває сторінку товару. Frontend перевіряє поле stock_quantity.

b. **Умова наявності:** Якщо товару немає, кнопка покупки блокується. Якщо товар є, кнопка активна.

c. **Перевірка авторизації:** Після спроби додати товар у кошик, система перевіряє наявність сесії. Неавторизованих клієнтів перенаправляють на сторінку реєстрації.

d. **Захист від накрутки:** Система перевіряє, чи ідентифікатор клієнта не збігається з ідентифікатором власника магазину.

e. **Оновлення кошика:** Якщо всі перевірки пройдено, товар додається до локального стану кошика, а лічильник в інтерфейсі оновлюється.

Алгоритм процесу "Призупинення сайту Адміністратором"

Це критичний алгоритм системної модерації, який демонструє автоматичне застосування санкцій до порушників.

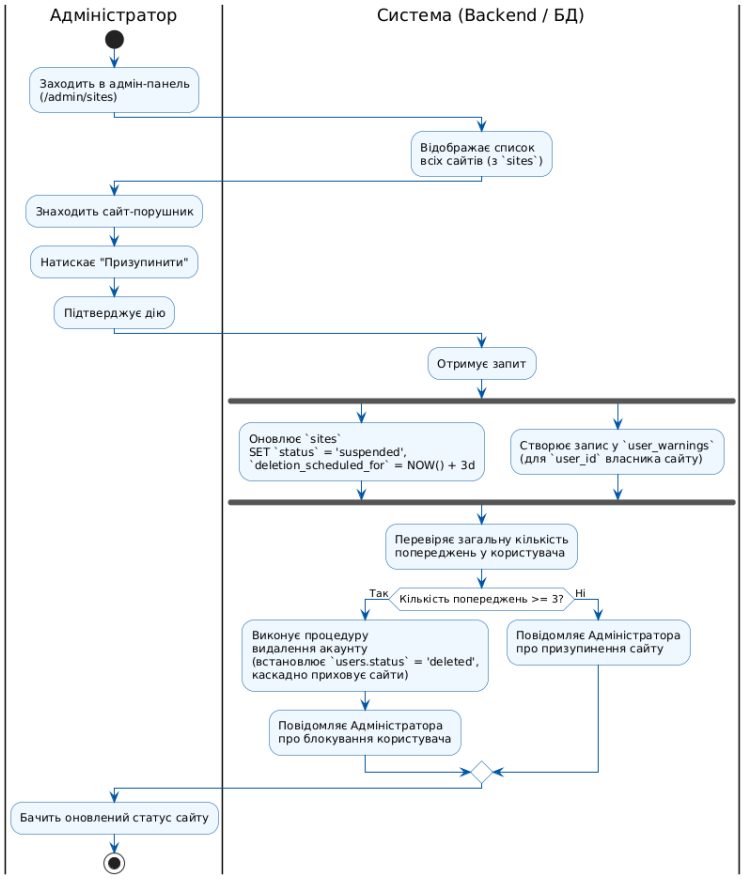

Рисунок 2.15 – Діаграма активності "Призупинення сайту"

Покроковий опис алгоритму:

a. **Ініціація:** Адміністратор ідентифікує сайт-порушник та ініціює призупинення.

b. **Паралельне виконання:** Backend одночасно змінює статус сайту на suspended та створює запис у таблиці user_warnings.

c. **Перевірка рецидивів:** Сервер підраховує загальну кількість попереджень у власника сайту.

d. **Прийняття рішення:** Якщо кількість попереджень досягає критичної межі (≥3), система автоматично блокує самого користувача та каскадно приховує всі його сайти. Якщо ліміт не перевищено – блокується лише конкретний сайт.

Алгоритм процесу "Обробка тікету підтримки"

Алгоритм ілюструє циклічний процес вирішення проблеми клієнта за допомогою системи тікетів.

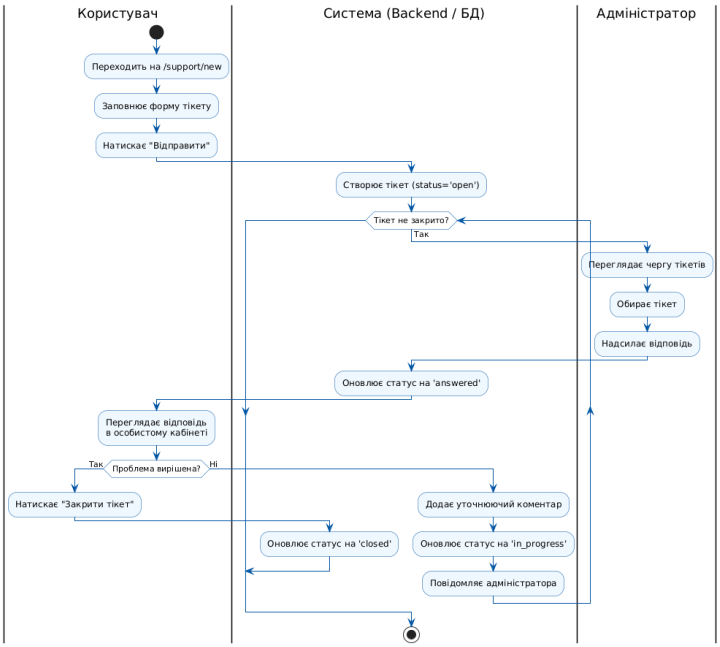

Рисунок 2.16 – Діаграма активності "Обробка тікету підтримки"

Покроковий опис алгоритму:

a. **Створення:** Користувач відправляє форму, Backend створює запис у таблиці support_tickets зі статусом open.

b. **Початок циклу:** Відкривається цикл перевірки while.

c. **Відповідь адміністратора:** Адміністратор надає відповідь, статус тікета змінюється на answered.

d. **Реакція користувача:** Користувач перевіряє відповідь. Якщо проблему вирішено, він підтверджує закриття і цикл розривається.

e. **Продовження циклу:** Якщо проблема не вирішена, користувач додає уточнення. Статус змінюється на in_progress, адміністратор отримує сповіщення, і цикл повертається на початок для повторної обробки.

Висновки до другого розділу

У другому розділі було виконано комплексне проєктування програмної архітектури платформи, яке охоплює всі етапи від розробки високорівневої моделі інфраструктури до деталізації алгоритмів взаємодії компонентів.

Побудовано діаграму розгортання, яка ілюструє мікросервісний підхід до побудови системи з використанням сучасних хмарних рішень. Такий розподіл гарантує відмовостійкість, глобальну доступність та незалежне масштабування клієнтської і серверної частин.

Проведено детальне моделювання варіантів використання системи, де визначено чотирьох незалежних акторів та формалізовано їхні права доступу. Для ключових прецедентів розроблено розширені специфікації, що слугують основою для контрактного програмування.

Розроблено об'єктно-орієнтовану структуру системи. Побудовані діаграми класів предметної області та класів реалізації ілюструють застосування класичних патернів проєктування. Діаграма компонентів відображає фізичну організацію коду за принципом Feature-Sliced Design для клієнтської частини та модульного поділу для серверної.

Спроєктовано структуру гібридної бази даних MySQL, нормалізованої до третьої нормальної форми, де реляційні зв’язки між системними сутностями поєднуються зі зберіганням користувацького контенту у форматі JSON.

Описано динамічну поведінку системи за допомогою діаграм активності, що деталізують бізнес-логіку ключових процесів. Крім того, побудовано діаграми послідовності та кооперації, які візуалізують хронологію та топологію передачі повідомлень між клієнтом, контролерами та шаром даних.

Розроблені логічні та структурні моделі повністю задовольняють вимоги технічного завдання і є вичерпною документацією для переходу до етапу безпосередньої фізичної реалізації програмного продукту, описаної у наступному розділі.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ

3.1 Програмна реалізація серверної частини та підсистеми безпеки

3.2 Розробка візуального редактора та клієнтського інтерфейсу

3.3 Тестування програмної системи

3.4 Керівництво користувача та розгортання платформи

Висновки до третього розділу

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТКИ

Додатку А
Вимоги

Додатку А
Глосарій

Додатку В
Структура проекту